

TABLE OF CONTENTS

Chapter No.	Chapter	Page No.
1	INTRODUCTION	2
1.1	Importance of Python in Solving Physics Problems	2
1.2	Scope of the Project	2
1.3	Objectives of the Project & formulas	2
2	COMPUTATIONAL TOOLS AND TECHNIQUES	13
2.1	PYTHON LIBRARIES USED	13
3	PYTHON IMPLEMENTATION FOR PHYSICS EXPERIMENTS	15
4	PYTHON SOURCE CODE FOR THE EXPERIMENTS	16
4.1	Standalone Python Program	16
4.2	Web-Based Python Programming	22
5	DIFFERENCE BETWEEN THE TWO CODES	30
5.1	User Interface	30
5.2	Input and Output Handling	30
5.3	Program Structure	30
5.4	Error Handling and Validation	31
5.5	Flexibility and Extensibility	31
5.6	Key Differences	31
6	OUTPUT OF TWO CODES	32
6.1	Standalone Python Program	32
6.2	Web-Based Python Programming	35
7	CONCLUSION	40
8	REFERENCES	41

CHAPTER 1:

INTRODUCTION

Physics experiments often involve complex calculations, data analysis, and graphical representations to interpret results accurately. Calculations are typically performed using a scientific calculator, which can be tiring, frustrating, and prone to errors, potentially delaying corrections. Delays in corrections may lead to mark deductions. Thus, Python programming simplifies the calculation process. This document explores how Python is utilized to solve numerical problems related to 20 physics experiments, covering topics such as optics, mechanics, electromagnetism, and materials science. Libraries such as NumPy, Matplotlib, Tabulate, and Streamlit were used.

1.1 IMPORTANCE OF PYTHON IN SOLVING PHYSICS PROBLEMS

Python has emerged as one of the most popular programming languages for scientific computing and data analysis. Its simplicity, readability, and extensive library ecosystem make it an ideal choice for solving numerical problems in physics. Key libraries such as NumPy (for numerical computations), Matplotlib (for data visualization), etc., provide powerful tools for analysing experimental data and solving complex equations. In this project, Python will be used to:

- Perform numerical calculations and simulations.
- Analyse and visualize experimental data.
- Implement mathematical models and algorithms.
- Validate theoretical predictions with experimental results.

1.2 SCOPE OF THE PROJECT

- This project covers a wide range of experiments, including:
- Optics: Polarimeter, Newton's Rings, Air Wedge.
- Electromagnetism: Calibration of a High-Range Voltmeter, Absolute Capacity of a Condenser, Self-Inductance of a Coil.
- Mechanics: Sonometer, Young's Modulus (Koeing's Method, Non-Uniform Bending, Stretching Method, Uniform Bending, Cantilever Method).
- Spectrometry: Spectrometer (Minimum Deviation, i-i' Curve, Small Angle Prism).
- Electrical Measurements: High Resistance by Leakage Method, Internal Resistance of a Cell, Mutual Inductance Methods.

1.3 OBJECTIVES OF THE PROJECT AND FORMULAE

The primary objectives of this project are:

- To apply numerical methods to solve problems arising from a variety of physics experiments.
- To develop Python-based solutions for data analysis, curve fitting, and error estimation.

- To demonstrate the practical applications of numerical methods in physics.
- To provide a comprehensive resource for students and researchers interested in solving physics problems using Python.

1.3.1 Polarimeter

Objective: To determine the specific rotation of a substance using a polarimeter.

Formula:

$$\rho = \theta / (C * 10 * l)$$

Where:

- ρ = Specific rotation (degrees $\text{dm}^{-1} \text{g}^{-1} \text{cm}^3$)
- θ = Angle of rotation (degrees)
- l = Path length (dm)
- C = Concentration (g/cm^3)

Implementation:

- Users input θ , l , and c .
- The program calculates α .

Results: The specific rotation α is displayed.

1.3.2 Calibration of a high-range voltmeter

Objective: To calibrate a voltmeter using a standard cell.

Formula:

$$V' = 50 * ((1.08 * l_2) / l_1)$$

Where:

- V' = Calculated Voltmeter reading
- l_2 = Balancing length when emf of Daniel cell is balanced
- l_1 = Balancing length which balances the voltage in a voltmeter
- V = observed voltmeter reading

Implementation:

- Users input E , R , and r .
- The program calculates V .

Results: The calibrated voltage V' is displayed.

1.3.3 Sonometer

Objective: To determine the frequency of a tuning fork using a sonometer.

Formula:

$$n = \frac{1}{2} * \sqrt{\left(\frac{M}{l^2}\right)(g/m)}$$

Where:

- n = Frequency (Hz)
- L = Length of the vibrating wire (m)
- M = Mass per unit length (kg/m)

Implementation:

- Users input L , and M .
- The program calculates n .

Results: The frequency n is displayed.

1.3.4 Absolute Capacity of a Condenser (BG Method)

Objective:

To determine the absolute capacity of a condenser using a ballistic galvanometer.

Formula:

$$C = \frac{T}{2\pi G} \left[\frac{d_1}{R^2} \right] \left[\frac{r^2}{d^2} \right] [1 + \lambda/2]$$

Where:

- R , r = Resistance

Implementation:

- The program calculates C .

Results: The capacitance C is displayed.

1.3.5 Astable Multivibrator

Objective: To determine the time period of an astable multivibrator.

Formula:

$$f = 1/1.386RC$$

Where:

- R = Resistor values (Ω)
- C = Capacitance (F)

Implementation:

- Users input R, and C.
- The program calculates f

Results: The time period T is displayed.

1.3.6 Newton's Rings

Objective: To determine the wavelength of monochromatic light using Newton's rings.

Formula:

$$R = (r_{n+m}^2 - r_n^2) / m \lambda$$

Where:

- λ = Wavelength (m)
- r_n, r_m = Diameters of nth and mth rings (m)
- R = Radius of curvature of the lens (m)

Implementation:

- Users input r_n, r_m, R, n , and m .
- The program calculates λ .

Results: The wavelength λ is displayed.

1.3.7 Air Wedge

Objective: To determine the thickness of a thin wire using the air wedge method.

Formula:

$$t = (\lambda l / 2\beta)m$$

Where:

- t = Thickness of wire (m)
- λ = Wavelength of light (m)
- β = Fringe width (m)

Implementation:

- Users input λ and fringe width.
- The program calculates t .

Results: The thickness t is displayed.

1.3.8 Spectrometer - minimum deviation

Objective: To determine the coefficient of friction using a sliding plane.

Formula:

$$\mu = (\sin(D+A)/2) / (\sin(A/2))$$

Where:

- μ = Refractive index of the material
- A = Angle of the prism
- D = Angle of minimum deviation

Implementation:

- Users input A and D .
- The program calculates μ .

Results: The coefficient of friction μ is displayed.

1.3.9 Spectrometer I-I' Curve

Objective:

To study the I-I' curve of a spectrometer.

Formula:

$$D = 2i - A, I = (D + A)/2$$

$$\mu = \sin i / \sin (A/2)$$

Implementation:

Users input measured angles.

The program plots the I-I' curve.

Results: The I-I' curve is displayed.

1.3.10 Small Angle Prism

Objective:

To determine the refractive index of a small angle prism.

Formula:

Normal Incidence method

$$\mu = \sin i' / \sin i = \sin (A + d) / \sin A$$

Normal Emergence Method

$$\mu = \sin i / \sin A$$

Minimum deviation Method

$$\mu = (\sin(D + A)/2) / (\sin(A/2))$$

Where:

- μ = Refractive index
- A = Prism angle (degrees)
- d = Minimum deviation angle (degrees)

Implementation:

- Users input A and d .
- The program calculates μ .

Results: The refractive index μ is displayed.

1.3.11 Young's Modulus (Uniform Bending)

Objective: To determine Young's modulus of a beam using the uniform bending method.

Formula:

$$Y = (g * l^3) / (4 * b * d^3)$$

Where:

- Y = Young's modulus (Pa)
- g = Acceleration due to gravity (m/s²)
- l = Length of beam (m)
- b = Breadth of beam (m)
- d = Thickness of beam (m)

Implementation:

- Users input l, b, d.
- The program calculates Y.

Results: Young's modulus Y is displayed.

1.3.12 Young's Modulus (Non-Uniform Bending)

Objective: To determine Young's modulus of a beam using the non-uniform bending method.

Formula:

$$Y = (m * g * l^3) / (b * d^3 * \Delta)$$

Where:

- Y = Young's modulus (Pa)
- m = Mass applied (kg)
- g = Acceleration due to gravity (m/s²)
- l = Length of beam (m)
- b = Breadth of beam (m)
- d = Thickness of beam (m)
- Δ = Deflection (m)

Implementation:

- Users input m, l, b, d, Δ .
- The program calculates Y.

Results: Young's modulus Y is displayed.

1.3.13 Young's Modulus (Cantilever)

Objective: To determine Young's modulus of a cantilever beam.

Formula:

$$Y = (4 * m * g * l^3) / (b * d^3 * \Delta)$$

Where:

- Y = Young's modulus (Pa)
- m = Mass applied (kg)
- g = Acceleration due to gravity (m/s²)
- l = Length of cantilever (m)
- b = Breadth of cantilever (m)
- d = Thickness of cantilever (m)
- Δ = Deflection (m)

Implementation:

- Users input m, l, b, d, Δ .
- The program calculates Y.

Results: Young's modulus Y is displayed.

1.3.14 Young's Modulus (Koenig's Method)

Objective: To determine Young's modulus using Koenig's method.

Formula:

$$Y = (4 * m * g * l^3) / (b * d^3 * \Delta) = (g * l^3) / (k * d^3)$$

Where:

- Y = Young's modulus (Pa)
- g = Acceleration due to gravity (m/s²)
- l = Length of beam (m)
- k = Constant (m)
- d = Thickness of beam (m)

Implementation:

- Users input l, k, d.
- The program calculates Y.

Results: Young's modulus Y is displayed.

1.3.15 Comparison of Mutual Inductance

Objective: To compare the mutual inductance of two coils.

Formula:

$$M1/M2 = (S1 + R1) / (S2 + R2)$$

Where:

- S1, S2 = Resistances of secondary coils
- R1, R2 = Resistance box values

Implementation:

- Users input S1, S2, R1, and R2.
- The program calculates the ratio M1/M2.

Results: The ratio of mutual inductances is displayed.

1.3.16 High Resistance by Leakage Method

Objective: To determine high resistance using the leakage method.

Formula:

$$R = (2.303 * C * \log_{10}(d1 / d2)) / t$$

Where:

- R = Resistance (Ω)
- C = Capacitance (F)
- d1, d2 = Distances (m)
- t = Time (s)

Implementation:

- Users input C, d1, d2, and t.

- The program calculates R.

Results: The resistance R is displayed.

1.3.17 Internal Resistance of a Cell Using BG

Objective: To determine the internal resistance of a cell using a ballistic galvanometer.

Formula:

$$B = R * ((d1 - d2) / d2)$$

Where:

- B = Internal resistance (Ω)
- R = Resistance box value (Ω)
- d1, d2 = Kicks (counts)

Implementation:

- Users input R, d1, and d2.
- The program calculates B.

Results: The internal resistance B is displayed.

1.3.18 Self-Inductance of a Coil (Anderson's Method)

Objective: To determine the self-inductance of a coil using Anderson's method.

Formula:

$$L = C \times (R \times Q + M \times (R + S))$$

Where:

- L = Self-inductance (H)
- C = Capacitance (F)
- Q, R, S, M = Resistance box values (Ω)

Implementation:

- Users input C, Q, R, S, and M.
- The program calculates L.

Results: The self-inductance L is displayed.

1.3.19 Self-Inductance of a Coil – Anderson’s Method

Objective: To determine the self-inductance of a coil using Anderson’s method.

Formula:

$$L = C \times (R \times Q + M \times (R + S))$$

Where:

- L = Self-inductance (H)
- C = Capacitance (F)
- Q, R, S, M = Resistance box values (Ω)

Implementation:

- Users input C, Q, R, S, and M.
- The program calculates L.

Results: The self-inductance L is displayed.

1.3.20 Self-Inductance of a Coil by Anderson Method

Objective: To determine the self-inductance of a coil using Anderson’s method.

Formula:

$$L = C \times (R \times Q + M \times (R + S))$$

Where:

- L = Self-inductance (H)
- C = Capacitance (F)
- Q, R, S, M = Resistance box values (Ω)

Implementation:

- Users input C, Q, R, S, and M.
- The program calculates L.

Results: The self-inductance L is displayed.

CHAPTER 2:

COMPUTATIONAL TOOLS AND TECHNIQUES

This project utilizes 20 Experiments from the experiments we have done in the laboratory, we have also used the PRACTICAL PHYSICS BOOK issued by the Department of Physics, ST. JOSEPH COLLEGE for reference, from which the formulas have been used. The Physics Experiments used in the programs are given below:

1. POLARIMETER
2. CALIBRATION OF A HIGH-RANGE VOLTMETER
3. SONOMETER
4. ABSOLUTE CAPACITY OF A CONDENSER- USING BG
5. ASTABLE MULTIVIBRATOR
6. AIR WEDGE- THICKNESS OF A WIRE
7. NEWTON'S RINGS
8. SPECTROMETER – MINIMUM DEVIATION
9. SPECTROMETER – i-i' CURVE
10. SPECTROMETER – SMALL ANGLE PRISM
11. YOUNG'S MODULUS – KEOING'S METHOD
12. YOUNG'S MODULUS – NON-UNIFORM BENDING
13. YOUNG'S MODULUS – STRECHING METHOD
14. YOUNG'S MODULUS – UNIFORMING BENDING
15. YOUNG'S MODULUS – CANTILEVER METHOD
16. ABSOLUTE MUTUAL INDUCTANCE METHOD
17. COMPARISON OF MUTUAL INDUCTANCE
18. HIGH RESISTANCE BY LEAKAGE METHOD
19. INTERNAL RESISTANCE OF A CELL USING BG
20. SELF-INDUCTANCE OF A COIL – ANDERSON'S METHOD

Firstly, each program to calculate the experiments were coded separately. And finally, when all the coding for all the experiments were finished, all 20 codes into a single code, which has two versions for, standalone text-based program and a web-based python program. Each now has over 700 lines.

2.1 PYTHON LIBRARIES USED

Library	Purpose
NumPy (NumPy)	Numerical calculations
Matplotlib (matplotlib.pyplot)	Graphing & visualization
Tabulate (tabulate)	Formatting tables
Streamlit (streamlit)	Easy to create Web-based UI

2.1.1 Numpy

- NumPy (Numerical Python) is a library in Python used for fast mathematical calculations.
- It helps with arrays, trigonometry, statistics, and matrix operations.
- It is faster than Python lists for numerical computations.

2.1.2 Matplotlib

- Matplotlib is a Python library used to create graphs and charts.
- It helps visualize experimental data in physics experiments.
- The main module used in Matplotlib is pyplot (imported as plt).

2.1.3 TABULATE

- tabulate is a Python library used to format numerical data into tables.
- It is useful for displaying experimental results in a structured format.
- Makes numerical data readable for reports.
- Easier than printing lists manually.
- Supports different table formats, e.g., "grid", "fancy grid", "html".

2.1.4 STREAMLIT

- Streamlit is a Python framework used to create interactive web apps.
- It allows real-time input and results without restarting the script.
- Best for physics experiments where users enter values & get instant results.
- This opens the app in a web browser (<http://localhost:8501>).
- Streamlit is an open-source Python library that allows you to create interactive web applications for data science and machine learning with minimal effort.
- It provides a simple and intuitive API to turn Python scripts into web apps without needing advanced web development skills.

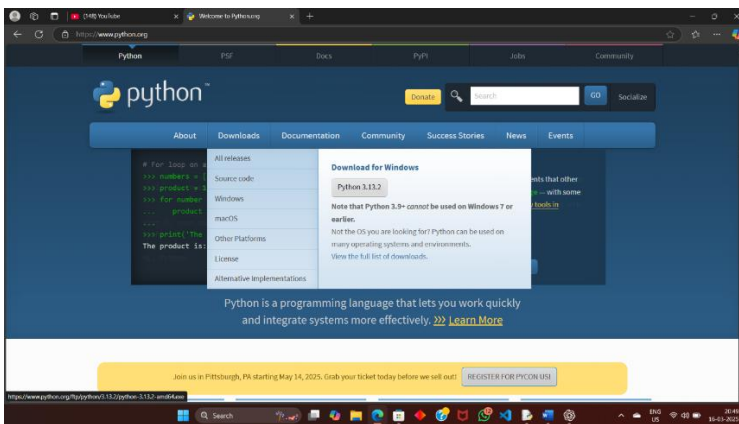
CHAPTER 3:

PYTHON IMPLEMENTATION FOR PHYSICS EXPERIMENTS

INSTALLING REQUIRED PYTHON AND ITS LIBRARIES

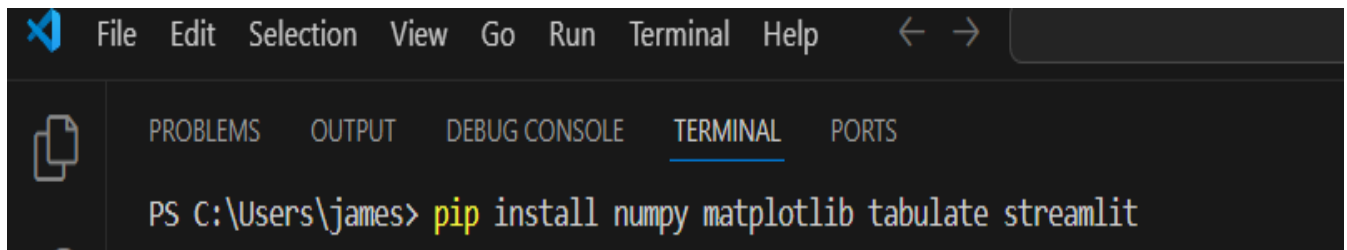
Python and its libraries must be installed, for that two apps can be used to run the program. Python can be downloaded from the official website, where the latest version is available. Go to python.org.

Inside click Downloads, inside this you can find the latest python version 3.13.2. On clicking that you will download the app



Or you can download Visual Studio code (VS code) which is available in Microsoft store. Once installed, open VS code, go to extensions (ctrl + shift+ X) and search for python and install.

Now to install its libraries, open command prompt or use terminal in Vs code.



Thus, the libraries are installed.

Once libraries are installed, it can be used to write program by import it into your code by simply writing 'import NumPy', will import the NumPy library.

CHAPTER 4:

PYTHON SOURCE CODE FOR THE EXPERIMENTS

Two versions of the source code were developed. The first is a standalone Python program for local execution, The second version is similar to the first but modified as an interactive web-based program.

4.1 STANDALONE PYTHON PROGRAM

```
import math
import numpy as np
import matplotlib.pyplot as plt
from tabulate import tabulate

def dms_to_decimal(deg, minutes):
    """Convert degrees and minutes to decimal degrees."""
    return deg + (minutes / 60)

def decimal_to_dms(decimal_degrees):
    """Convert decimal degrees to degrees, minutes, and properly
    rounded seconds."""
    degrees = int(decimal_degrees)
    minutes_decimal = (decimal_degrees - degrees) * 60
    minutes = int(minutes_decimal)
    seconds = round((minutes_decimal - minutes) * 60)
    if seconds == 60:
        seconds = 0
        minutes += 1

    if minutes == 60:
        minutes = 0
        degrees += 1

    return degrees, minutes, seconds

def calculate_value(theta_deg, theta_min, l):
    """Calculate  $\theta / (10 \times l)$  and return result in DMS format."""
    theta_decimal = dms_to_decimal(theta_deg, theta_min)
    result_decimal = theta_decimal / (10 * l)
    return decimal_to_dms(result_decimal)

def calculate_error_percentage(standard_deg, standard_min,
observed_deg, observed_min):
    """Calculate error percentage given standard and observed
    values in degrees and minutes."""
    standard_decimal = dms_to_decimal(standard_deg,
standard_min)
    observed_decimal = dms_to_decimal(observed_deg,
observed_min)

    error_percentage = abs((standard_decimal - observed_decimal) /
standard_decimal) * 100
    return round(error_percentage, 2)

def program_1():
    # inputs
    theta_deg = int(input("Enter  $\theta$ /c degrees: "))
    theta_min = float(input("Enter  $\theta$ /c minutes: "))
    l = float(input("Enter l (in meter): "))

    result_d, result_m, result_s = calculate_value(theta_deg,
theta_min, l)
    print(f"Calculated Value: {result_d}° {result_m}' {result_s}\"")

    # Error calculation
    standard_deg = int(input("Enter standard value degrees: "))
    standard_min = float(input("Enter standard value minutes: "))
    observed_deg = int(input("Enter observed value degrees: "))
    observed_min = float(input("Enter observed value minutes: "))

    error_percent = calculate_error_percentage(standard_deg,
standard_min, observed_deg, observed_min)
    print(f"Error Percentage: {error_percent}%")

def program_2():
    # Input: Number of times to calculate
    n = int(input("Enter the number of calculations: "))

    # Enter l1 once
    l1 = float(input("Enter l1: "))

    # Lists to store values
    V_list = []
    l2_list = []
    V_prime_list = []
    correction_list = []

    # Loop to calculate V' and correction
    for i in range(n):
        print(f"\n--- Calculation {i+1} ---")
        V = float(input("Enter V: ")) # Get V from user
        l2 = float(input("Enter l2: ")) # Get l2 from user

        V_prime = 50 * 1.08 * l2 / l1 # Formula for V'
        correction = V_prime - V # Correction calculation

    # Store values in lists
    V_list.append(V)
    l2_list.append(l2)
    V_prime_list.append(V_prime)
    correction_list.append(correction)

    # Create tabular data
    table_data = []
    for i in range(n):
        table_data.append([V_list[i], l2_list[i], V_prime_list[i],
correction_list[i]])

    # Print table
    headers = ["Voltmeter readings", "l2", "V' (Calculated
Voltage)", "Correction (V' - V)"]
    print("\nResults:")
```



```

print(tabulate(table_data, headers=headers, tablefmt="grid"))

# *Graph 1: V vs. Correction (V' - V)*
plt.figure(figsize=(8, 5))
plt.scatter(V_list, correction_list, color='red', marker='o',
label="Correction (V' - V)")
plt.plot(V_list, correction_list, linestyle='--', alpha=0.7,
color='red')
plt.xlabel("V")
plt.ylabel("V' - V (Correction)")
plt.title("Graph 1: V vs. Correction")
plt.legend()
plt.grid(True)

# *Graph 2: V vs. V''*
plt.figure(figsize=(8, 5))
plt.scatter(V_list, V_prime_list, color='blue', marker='o',
label="V'")
plt.plot(V_list, V_prime_list, linestyle='--', alpha=0.7,
color='blue')
plt.xlabel("V")
plt.ylabel("V'")
plt.title("Graph 2: V vs. V'")
plt.legend()
plt.grid(True)

# Show the graphs
plt.show()

def program_3():
    def calculate_mass_of_wire(r, rho):
        """Calculate mass per unit length (m) using  $m = \pi(r^2)\rho$ ."""
        return math.pi * (r ** 2) * rho

    def calculate_frequency(M, g, r, rho):
        """Calculate the frequency of the tuning fork."""
        mass_of_wire = calculate_mass_of_wire(r, rho)
        frequency = (1 / 2) * math.sqrt((M * g) / mass_of_wire)
        return frequency

    # Taking user input
    M = eval(input("Enter the mass/length^2(M/l^2) in kg/m^2: "))
    g = 9.81 # Acceleration due to gravity in m/s^2
    r = float(input("Enter the radius of the wire (r) in meters: "))
    rho = float(input("Enter the density of the wire (ρ) in kg/m^3:
"))

    # Calculating frequency
    frequency = calculate_frequency(M, g, r, rho)

    # Display result
    print(f"Frequency of the tuning fork: {frequency:.2f} Hz")

def program_4():
    def calculate_C(T, d1_R2, r2_d2, G, a1, a11):
        # Calculate lambda and round to 4 decimal places
        lambda_value = round((2.303 / 10) * math.log10(a1 / a11), 4)

        # Calculate C in Farads (keeping full precision)
        C = (T / (2 * math.pi * G)) * d1_R2 * r2_d2 * (1 +
(lambda_value / 2))

        # Convert C to microfarads (1 F = 10^6 μF) before rounding
        C_microfarad = round(C * 1e6, 4)

        return lambda_value, C_microfarad

    # Example usage
    T = float(input("Enter T: "))
    d1_R2 = eval(input("Enter d1/R2: "))
    r2_d2 = float(input("Enter r2/d2: "))
    G = float(input("Enter G: "))
    a1 = float(input("Enter a1: "))
    a11 = float(input("Enter a11: "))

    lambda_value, C_value = calculate_C(T, d1_R2, r2_d2, G, a1,
a11)

    # Display results
    print(f"λ (lambda): {lambda_value}")
    print(f"C (Capacitance): {C_value} μF")

def program_5():
    def calculate_frequency(R, C):
        return 1 / (1.386 * R * C)

    # Ask for the number of repetitions
    n = int(input("Enter number of times you wanna calculate: "))

    # Run the loop for n times
    for i in range(n):
        print(f"\nCalculation {i + 1} of {n}:")

        # Taking user inputs with support for scientific notation
        R = eval(input("Enter resistance (R) in ohms: "))
        C = eval(input("Enter capacitance (C) in farads: "))

        # Calculate and display frequency
        frequency = calculate_frequency(R, C)
        print(f"Frequency: {frequency:.6f} Hz")

def program_6():
    # Function to compute the thickness of the wire
    def compute_thickness(lambda_val, l, beta, m):
        return (lambda_val * l * m) / (2 * beta)

    # Constants
    lambda_val = 5893e-10 # Wavelength in meters (5893 Å)

    while True:
        try:
            # User inputs
            l = float(input("Enter the distance between the wire (l) in
meters: ").strip())
            beta = float(input("Enter the width of the ring (β) in meters:
").strip())
            m = int(input("Enter the number of rings (m): ").strip())

            # Validate inputs
            if l <= 0 or beta <= 0 or m <= 0:
                raise ValueError("All values must be positive numbers.")

            # Compute thickness
            thickness = compute_thickness(lambda_val, l, beta, m)

            # Display output
            print("\n--- Results ---")
            print(f"Wavelength (λ): {lambda_val} meters")
            print(f"Distance between wire (l): {l} meters")
            print(f"Width of ring (β): {beta} meters")

```

```

    print(f"Number of rings (m): {m}")
    print(f"Calculated thickness of wire (t): {thickness:.6e}
meters\n")

except ValueError as e:
    print(f"Error: {e}")
except Exception as e:
    print(f"An unexpected error occurred: {e}")

# Ask if the user wants to continue
choice = input("Do you want to compute another value?
(yes/no): ").strip().lower()
if choice not in ["yes", "y"]:
    print("Exiting program. Goodbye!")
    break

def program_7():
    def compute_R(r_n, r_nm, m, wavelength):
        return (r_nm**2 - r_n**2) / (m * wavelength)

    # Given wavelength in meters (5893 Å)
    wavelength = 5893e-10 # 5893 Å in meters

    while True:
        try:
            # Taking input values
            r_n = float(input("Enter the radius of the nth ring in meters:
").strip())
            r_nm = float(input("Enter the radius of the (n+m)th ring in
meters: ").strip())
            m = int(input("Enter the number of rings after nth ring (m):
").strip())

            # Validations
            if r_nm <= 0 or r_n <= 0 or m <= 0:
                print("Error: Radii and m must be positive values. Try
again.")
                continue
            if r_nm <= r_n:
                print("Error: r_nm must be greater than r_n. Try again.")
                continue

            # Compute R
            R_value = compute_R(r_n, r_nm, m, wavelength)

            # Print formatted output
            print(f"Computed R value: {R_value:.5e} m-1")

        except ValueError:
            print("Error: Please enter valid numerical values. Try
again.")
            continue

        # Ask if the user wants to continue
        choice = input("Do you want to compute another value?
(yes/no): ").strip().lower()
        if choice not in ["yes", "y"]:
            print("Exiting program. Goodbye!")
            break

def program_8():
    # Function to convert degrees and minutes to decimal degrees
    def convert_to_decimal(degrees, minutes):
        return degrees + (minutes / 60)

```

```

    # Function to compute the coefficient of friction (mu)
    def calculate_coefficient_of_friction(D_deg, D_min, A_deg,
A_min):
        # Convert D and A to decimal degrees
        D_value = convert_to_decimal(D_deg, D_min)
        A_value = convert_to_decimal(A_deg, A_min)

        if A_value == 0:
            raise ValueError("A cannot be zero to avoid division by
zero.")

        # Convert degrees to radians for calculations
        D_radians = math.radians(D_value)
        A_radians = math.radians(A_value)

        # Apply the formula
        mu = math.sin((D_radians + A_radians) / 2) /
math.sin(A_radians / 2)
        return mu

    # Main script
    try:
        n = int(input("Enter the number of calculations: "))

        results = []
        for i in range(n):
            print(f"\n--- Calculation {i+1} ---")
            try:
                # Get degrees and minutes for D
                D_deg = float(input(f"Enter the degrees for D (set
{i+1}): "))
                D_min = float(input(f"Enter the minutes for D (set
{i+1}): "))

                # Get degrees and minutes for A
                A_deg = float(input(f"Enter the degrees for A (set
{i+1}): "))
                A_min = float(input(f"Enter the minutes for A (set
{i+1}): "))

                mu_result = calculate_coefficient_of_friction(D_deg,
D_min, A_deg, A_min)
                results.append((D_deg, D_min, A_deg, A_min,
mu_result))

            except ValueError as e:
                print(f"Invalid input: {e}")

        # Display results with breaks
        print("\n===== Final Results =====")
        for i, (D_d, D_m, A_d, A_m, mu) in enumerate(results, 1):
            print(f"\nCalculation {i}:")
            print(f"  D = {D_d}° {D_m}'")
            print(f"  A = {A_d}° {A_m}'")
            print(f"  -> mu = {mu:.6f}")
            print("-" * 30) # Adds a separator line

        except ValueError:
            print("Invalid input. Please enter a valid integer for the
number of calculations.")

def program_9():
    # Function to convert degrees and minutes to decimal degrees
    def deg_min_to_decimal(degrees, minutes):
        return degrees + (minutes / 60)

```

```

# Function to compute refractive index using the Normal
Incidence Method
def normal_incidence_method(A, d):
    A_rad = math.radians(A)
    d_rad = math.radians(d)
    return math.sin(A_rad + d_rad) / math.sin(A_rad)

# Function to compute refractive index using the Normal
Emergence Method
def normal_emergence_method(i, A):
    i_rad = math.radians(i)
    A_rad = math.radians(A)
    return math.sin(i_rad) / math.sin(A_rad)

# Function to compute refractive index using the Minimum
Deviation Method
def minimum_deviation_method(D, A):
    D_rad = math.radians(D)
    A_rad = math.radians(A)
    return math.sin((D_rad + A_rad) / 2) / math.sin(A_rad / 2)

# Main program loop
while True:
    print("\nChoose a method to calculate the refractive index:")
    print("1. Normal Incidence Method")
    print("2. Normal Emergence Method")
    print("3. Minimum Deviation Method")
    print("4. Exit")

    choice = input("Enter your choice (1/2/3/4): ").strip()

    if choice == "1":
        # Input for angle of prism (A)
        A_deg = int(input("Enter degrees for angle of prism (A):
"))
        A_min = float(input("Enter minutes for angle of prism (A):
"))
        A = deg_min_to_decimal(A_deg, A_min)

        # Input for angle of deviation (d)
        d_deg = int(input("Enter degrees for angle of deviation (d):
"))
        d_min = float(input("Enter minutes for angle of deviation
(d): "))
        d = deg_min_to_decimal(d_deg, d_min)

        mu = normal_incidence_method(A, d)
        print(f"Refractive Index ( $\mu$ ) using Normal Incidence
Method: {mu:.6f}")

    elif choice == "2":
        # Input for angle of incidence (i)
        i_deg = int(input("Enter degrees for angle of incidence (i):
"))
        i_min = float(input("Enter minutes for angle of incidence
(i): "))
        i = deg_min_to_decimal(i_deg, i_min)

        # Input for angle of prism (A)
        A_deg = int(input("Enter degrees for angle of prism (A):
"))
        A_min = float(input("Enter minutes for angle of prism (A):
"))
        A = deg_min_to_decimal(A_deg, A_min)

```

```

        mu = normal_emergence_method(i, A)
        print(f"Refractive Index ( $\mu$ ) using Normal Emergence
Method: {mu:.6f}")

    elif choice == "3":
        # Input for angle of minimum deviation (D)
        D_deg = int(input("Enter degrees for angle of minimum
deviation (D): "))
        D_min = float(input("Enter minutes for angle of minimum
deviation (D): "))
        D = deg_min_to_decimal(D_deg, D_min)

        # Input for angle of prism (A)
        A_deg = int(input("Enter degrees for angle of prism (A):
"))
        A_min = float(input("Enter minutes for angle of prism (A):
"))
        A = deg_min_to_decimal(A_deg, A_min)

        mu = minimum_deviation_method(D, A)
        print(f"Refractive Index ( $\mu$ ) using Minimum Deviation
Method: {mu:.6f}")

    elif choice == "4":
        print("Exiting program. Goodbye!")
        break

    else:
        print("Invalid choice! Please enter 1, 2, 3, or 4.")

# Ask if the user wants to continue
choice = input("\nDo you want to compute another value?
(yes/no): ").strip().lower()
if choice not in ["yes", "y"]:
    print("Exiting program. Goodbye!")
    break

def program_10():
    def dms_to_decimal(degrees, minutes):
        return degrees + (minutes / 60)

    def calculate_mu(D, A):
        i = (D + A) / 2 # Compute i
        A_rad = np.radians(A) # Convert A to radians
        i_rad = np.radians(i) # Convert i to radians

        if np.sin(A_rad / 2) == 0:
            return np.nan # Avoid division by zero

        mu = np.sin(i_rad) / np.sin(A_rad / 2) # Compute mu
        return mu

# User will input multiple values for D and A in a single entry
n = int(input("Enter the number of values: "))
D_values = []
A_values = []
mu_values = []

print("Enter values in the format: degrees minutes ")

for i in range(n):
    D_deg, D_min = map(float, input(f"Enter D (Deviation
Angle) for set {i+1}: ").split())

```

```

    A_deg, A_min = map(float, input(f"Enter A (Angle of Prism)
for set {i+1}: ").split())

```

```

    D = dms_to_decimal(D_deg, D_min)
    A = dms_to_decimal(A_deg, A_min)

```

```

    mu = calculate_mu(D, A)

```

```

    D_values.append(D)
    A_values.append(A)
    mu_values.append(mu)

```

```

    print(f"Calculated Refractive Index ( $\mu$ ) for set {i+1}: {mu}")

```

```

# Plot the results
plt.figure(figsize=(5, 5))
plt.scatter(D_values, mu_values, color='r', label='μ values')
plt.xlabel('D (Deviation Angle in Degrees)')
plt.ylabel('μ (Refractive Index)')
plt.title(f'Refractive Index Calculation for {n} Values')
plt.legend()
plt.grid()
plt.show()

```

```

def program_11():
    def youngs_modulus_koenig(frequency, length, width, height,
density):
        # Second moment of area for a rectangular beam ( $I = b * h^3 /$ 
12)

```

```

        I = (width * height**3) / 12

```

```

        # Cross-sectional area of the beam ( $A = b * h$ )
        A = width * height

```

```

        # Calculating Young's Modulus using Koenig's method
formula
        E = (2 * math.pi * frequency)**2 * (density * A * length**4) / I

```

```

        return E

```

```

# Input values
frequency = float(input("Enter the frequency of vibration (Hz):
"))
length = float(input("Enter the length of the beam (m): "))
width = float(input("Enter the width of the beam (m): "))
height = float(input("Enter the height of the beam (m): "))
density = float(input("Enter the density of the material (kg/m³):
"))

```

```

# Calculate Young's Modulus
E = youngs_modulus_koenig(frequency, length, width, height,
density)

```

```

# Output the result
print(f"The Young's Modulus of the material is: {E:.2e} Pa")

```

```

def program_12():
    def youngs_modulus(force, original_length, change_in_length,
area):

```

```

        # Calculate stress
        stress = force / area

```

```

        # Calculate strain
        strain = change_in_length / original_length

```

```

        # Calculate Young's modulus
        E = stress / strain
        return E

```

```

# Example values
force = float(input("Enter applied force (N): "))
original_length = float(input("Enter original length (m): "))
change_in_length = float(input("Enter change in length (m): "))
area = float(input("Enter cross-sectional area (m²): "))

```

```

# Calculate and display Young's modulus
E = youngs_modulus(force, original_length, change_in_length,
area)
print(f"Young's Modulus: {E} Pa")

```

```

def program_13():
    def youngs_modulus(F, L, A, delta_L):
        if delta_L == 0:
            return "Change in length cannot be zero"

```

```

        E = (F * L) / (A * delta_L)
        return E

```

```

# Example values
F = float(input("Enter the applied force (N): "))
L = float(input("Enter the original length (m): "))
A = float(input("Enter the cross-sectional area (m²): "))
delta_L = float(input("Enter the change in length (m): "))

```

```

E = youngs_modulus(F, L, A, delta_L)

```

```

print(f"Young's Modulus: {E:.2e} Pa") # Scientific notation
format

```

```

def program_14():
    def youngs_modulus_uniform_bending():
        print("Young's Modulus Calculation - Uniform Bending")

```

```

# Input values
m = float(input("Enter mass (kg): "))
L = float(input("Enter length of the beam (m): "))
b = float(input("Enter breadth of the beam (m): "))
d = float(input("Enter thickness of the beam (m): "))
y = float(input("Enter depression (m): "))

```

```

g = 9.81 # Acceleration due to gravity (m/s²)

```

```

# Calculation
Y = (m * g * L**3) / (4 * b * d**3 * y)

```

```

# Display result
print(f"Young's Modulus (Y) = {Y:.2e} N/m²")

```

```

# Run the function
if __name__ == "__main__":
    youngs_modulus_uniform_bending()

```

```

def program_15():
    def youngs_modulus_cantilever(L, F, b, d, delta):
        E = (4 * L**3 * F) / (b * d**3 * delta)
        return E

```

```

# Input values
L = float(input("Enter the length of the cantilever (m): "))
F = float(input("Enter the applied force (N): "))

```

```

b = float(input("Enter the width of the beam (m): "))
d = float(input("Enter the thickness (depth) of the beam (m): "))
delta = float(input("Enter the deflection (m): "))

# Calculate Young's modulus
E = youngs_modulus_cantilever(L, F, b, d, delta)

print(f"Young's modulus (E) = {E:.2e}")

def program_16():
    def calculate_lambda(a1, a11):
        return 2.303 * math.log(a1 / a11)

    def calculate_M(r, T, a1, a11, d1, d2):
        lambda_value = calculate_lambda(a1, a11)
        M = (r * T / (2 * math.pi)) * (1 + lambda_value / 2) * (d1 /
d2)
        return M

    while True:

        r = float(input("Enter the fractional resistance (r): "))
        T = float(input("Enter the time in seconds (T): "))
        a1 = float(input("Enter the maximum distance from center for
one oscillation (a1): "))
        a11 = float(input("Enter the maximum distance from center
for the eleventh oscillation (a11): "))
        d1 = float(input("Enter the first kick in meters (d1): "))
        d2 = float(input("Enter the second kick in meters (d2): "))

        M = calculate_M(r, T, a1, a11, d1, d2)
        print(f"The absolute mutual inductance (M) is: {M}")

        again = input("Press 'Enter' to run again or type 'exit' to stop:
").strip()
        if again.lower() == 'exit':
            break

def program_17():
    def calculate_mutual_inductance_ratio(S1, S2, R1, R2):
        M1_M2_ratio = (S1 + R1) / (S2 + R2)
        return M1_M2_ratio

    while True:

        S1 = float(input("Enter the resistance of the secondary coil S1
(in ohms): "))
        S2 = float(input("Enter the resistance of the secondary coil S2
(in ohms): "))
        R1 = float(input("Enter the resistance box value R1 (in ohms):
"))
        R2 = float(input("Enter the resistance box value R2 (in ohms):
"))

        ratio = calculate_mutual_inductance_ratio(S1, S2, R1, R2)
        print(f"The ratio of M1 to M2 is: {ratio}")

        again = input("Press 'Enter' to run again or type 'exit' to stop:
").strip()
        if again.lower() == 'exit':
            break

def program_18():
    def calculate_resistance(C_microfarads, d1_meters, d2_meters,
t_seconds):
        C_farads = C_microfarads * 1e-6

        log_term = math.log10(d1_meters / d2_meters)

        R_ohms = t_seconds / (2.303 * C_farads * log_term)

        R_mega_ohms = R_ohms * 1e-6

        return R_mega_ohms

    C_microfarads = float(input("Enter capacitance (C) in
microfarads: "))
    d1_meters = float(input("Enter distance (d1) in meters: "))
    d2_meters = float(input("Enter distance (d2) in meters: "))
    t_seconds = float(input("Enter time (t) in seconds: "))

    R_mega_ohms = calculate_resistance(C_microfarads,
d1_meters, d2_meters, t_seconds)
    print(f"Resistance R: {R_mega_ohms} 10^6 (M)ohms")

def program_19():
    def calculate_internal_resistance(R, d1, d2):
        B = R * (d1 - d2) / d2
        return B

    while True:
        # Get input values from the user
        R = float(input("Enter the resistance box value (R): "))
        d1 = float(input("Enter the kick in distance d1 (in meters): "))
        d2 = float(input("Enter the kick in distance d2 (in meters): "))

        # Calculate and display the internal resistance B
        B = calculate_internal_resistance(R, d1, d2)
        print(f"The internal resistance of the cell (B) is: {B}")

        # Ask the user if they want to run the code again
        again = input("Press 'Enter' to run again or type 'exit' to stop:
").strip()
        if again.lower() == 'exit':
            break

def program_20():
    def calculate_self_induction(C, Q, R, S, M):
        L = C * (R * Q + M * (R + S))
        return L

    while True:

        C = float(input("Enter the condenser value (C) in microfarads:
")) * 10**(-6)
        Q = float(input("Enter the resistance box value Q (in ohms):
"))
        R = float(input("Enter the resistance box value R (in ohms):
"))
        S = float(input("Enter the resistance box value S (in ohms):
"))
        M = float(input("Enter the resistance box value M (in ohms):
"))

        L = calculate_self_induction(C, Q, R, S, M)
        print(f"The self-induction L is: {L} Henry")

```

```

        again = input("Press 'Enter' to run again or type 'exit' to stop:
").strip()
        if again.lower() == 'exit':
            break

```

```

def main():
    print("Select a program to run:")
    print("1. Spectrometer")
    print("2. Calibration of a high range voltmeter")
    print("3. Sonometer")
    print("4. BG absolute capacity of a condenser")
    print("5. Astable multivibrator")
    print("6. Air wedge")
    print("7. Newton rings")
    print("8. Minimum deviation")
    print("9. Small angle prism")
    print("10. Spectrometer i-i' curve")
    print("11. Young's modulus (Koenig method)")
    print("12. Young's modulus (Non-uniform bending)")
    print("13. Young's modulus (Stretching method)")
    print("14. Young's modulus (Uniform bending)")
    print("15. Young's modulus (Cantilever)")
    print("16. Absolute mutual inductance")
    print("17. Comparison of mutual inductance")
    print("18. High resistance by leakage method")
    print("19. Internal resistance of a cell BG")
    print("20. Self inductance of a coil by Anderson method")

    choice = int(input("Enter the number of the program you want to
run: "))

```

```

    if choice == 1:
        program_1()
    elif choice == 2:
        program_2()
    elif choice == 3:
        program_3()
    elif choice == 4:

```

```

        program_4()
    elif choice == 5:
        program_5()
    elif choice == 6:
        program_6()
    elif choice == 7:
        program_7()
    elif choice == 8:
        program_8()
    elif choice == 9:
        program_9()
    elif choice == 10:
        program_10()
    elif choice == 11:
        program_11()
    elif choice == 12:
        program_12()
    elif choice == 13:
        program_13()
    elif choice == 14:
        program_14()
    elif choice == 15:
        program_15()
    elif choice == 16:
        program_16()
    elif choice == 17:
        program_17()
    elif choice == 18:
        program_18()
    elif choice == 19:
        program_19()
    elif choice == 20:
        program_20()
    else:
        print("Invalid choice. Please select a number between 1 and
20.")

if __name__ == "__main__":
    main()

```

This is the source code for text based python based program.

4.2 WEB-BASED PYTHON PROGRAMMING

This uses the same code that is given above but has little alteration to be done. This cannot be runned in a python compiler, so it uses Streamlit library to to create an interactive web application for solving the same problems.

This is the source code for web-based python solver.

```

import streamlit as st
import math
import numpy as np
import matplotlib.pyplot as plt
from tabulate import tabulate

# Helper functions
def dms_to_decimal(deg, minutes):
    """Convert degrees and minutes to decimal degrees."""
    return deg + (minutes / 60)

```

```

def decimal_to_dms(decimal_degrees):
    """Convert decimal degrees to degrees, minutes, and
properly rounded seconds."""
    degrees = int(decimal_degrees)
    minutes_decimal = (decimal_degrees - degrees) * 60
    minutes = int(minutes_decimal)
    seconds = round((minutes_decimal - minutes) * 60)
    if seconds == 60:
        seconds = 0
        minutes += 1
    if minutes == 60:

```

```

        minutes = 0
        degrees += 1
    return degrees, minutes, seconds

def calculate_value(theta_deg, theta_min, l):
    """Calculate  $(\theta) / (10 \times l)$  and return result in DMS
    format."""
    theta_decimal = dms_to_decimal(theta_deg, theta_min)
    result_decimal = theta_decimal / (10 * l)
    return decimal_to_dms(result_decimal)

def calculate_error_percentage(standard_deg,
standard_min, observed_deg, observed_min):
    """Calculate error percentage given standard and
    observed values in degrees and minutes."""
    standard_decimal = dms_to_decimal(standard_deg,
standard_min)
    observed_decimal = dms_to_decimal(observed_deg,
observed_min)
    error_percentage = abs((standard_decimal -
observed_decimal) / standard_decimal) * 100
    return round(error_percentage, 2)

# Program 1: Spectrometer
def program_1():
    st.header("Program 1: Polarimeter")
    theta_deg = st.number_input("Enter  $\theta$ /c degrees:",
value=0)
    theta_min = st.number_input("Enter  $\theta$ /c minutes:",
value=0.0)
    l = st.number_input("Enter l (in meter):", value=1.0)

    if st.button("Calculate Value"):
        result_d, result_m, result_s =
calculate_value(theta_deg, theta_min, l)
        st.success(f"Calculated Value: {result_d}° {result_m}'
{result_s}\"")

    # Error calculation
    st.subheader("Error Calculation")
    standard_deg = st.number_input("Enter standard value
degrees:", value=0)
    standard_min = st.number_input("Enter standard value
minutes:", value=0.0)
    observed_deg = st.number_input("Enter observed value
degrees:", value=0)
    observed_min = st.number_input("Enter observed value
minutes:", value=0.0)

    if st.button("Calculate Error Percentage"):
        error_percent =
calculate_error_percentage(standard_deg, standard_min,
observed_deg, observed_min)
        st.success(f"Error Percentage: {error_percent}%")

# Program 2: Calibration of a high range voltmeter
def program_2():

```

```

    st.header("Program 2: Calibration of a High Range
Voltmeter")
    n = st.number_input("Enter the number of calculations:",
value=1, min_value=1)
    l1 = st.number_input("Enter l1:", value=1.0)

    V_list = []
    l2_list = []
    V_prime_list = []
    correction_list = []

    for i in range(n):
        st.subheader(f"Calculation {i + 1}")
        V = st.number_input(f"Enter V for calculation {i +
1}:", value=0.0)
        l2 = st.number_input(f"Enter l2 for calculation {i +
1}:", value=0.0)

        V_prime = 50 * 1.08 * l2 / l1
        correction = V_prime - V

        V_list.append(V)
        l2_list.append(l2)
        V_prime_list.append(V_prime)
        correction_list.append(correction)

    if st.button("Generate Results"):
        table_data = []
        for i in range(n):
            table_data.append([V_list[i], l2_list[i],
V_prime_list[i], correction_list[i]])

        headers = ["Voltmeter readings", "l2", "V' (Calculated
Voltage)", "Correction (V' - V)"]
        st.table(tabulate(table_data, headers=headers,
tablefmt="grid"))

    # Plot graphs
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

    # Graph 1: V vs. Correction (V' - V)
    ax1.scatter(V_list, correction_list, color='red',
marker='o', label="Correction (V' - V)")
    ax1.plot(V_list, correction_list, linestyle='--',
alpha=0.7, color='red')
    ax1.set_xlabel("V")
    ax1.set_ylabel("V' - V (Correction)")
    ax1.set_title("Graph 1: V vs. Correction")
    ax1.legend()
    ax1.grid(True)

    # Graph 2: V vs. V'
    ax2.scatter(V_list, V_prime_list, color='blue',
marker='o', label="V'")
    ax2.plot(V_list, V_prime_list, linestyle='--',
alpha=0.7, color='blue')
    ax2.set_xlabel("V")
    ax2.set_ylabel("V'")

```

```

ax2.set_title("Graph 2: V vs. V'")
ax2.legend()
ax2.grid(True)

st.pyplot(fig)

# Program 3: Sonometer
def program_3():
    st.header("Program 3: Sonometer")

    def calculate_mass_of_wire(r, rho):
        """Calculate mass per unit length (m) using  $m = \pi(r^2)\rho$ ."""
        return math.pi * (r ** 2) * rho

    def calculate_frequency(M, g, r, rho):
        """Calculate the frequency of the tuning fork."""
        mass_of_wire = calculate_mass_of_wire(r, rho)
        frequency = (1 / 2) * math.sqrt((M * g) / mass_of_wire)
        return frequency

    M = st.number_input("Enter the mass/length^2 (M/l^2) in kg/m^2:", value=0.0)
    g = 9.81 # Acceleration due to gravity in m/s^2
    r = st.number_input("Enter the radius of the wire (r) in meters:", value=0.0)
    rho = st.number_input("Enter the density of the wire ( $\rho$ ) in kg/m^3:", value=0.0)

    if st.button("Calculate Frequency"):
        frequency = calculate_frequency(M, g, r, rho)
        st.success(f"Frequency of the tuning fork: {frequency:.2f} Hz")

# Program 4: BG absolute capacity of a condenser
def program_4():
    st.header("Program 4: BG Absolute Capacity of a Condenser")

    def calculate_C(T, d1_R2, r2_d2, G, a1, a11):
        lambda_value = round((2.303 / 10) * math.log10(a1 / a11), 4)
        C = (T / (2 * math.pi * G)) * d1_R2 * r2_d2 * (1 + (lambda_value / 2))
        C_microfarad = round(C * 1e6, 4)
        return lambda_value, C_microfarad

    T = st.number_input("Enter T:", value=0.0)
    d1_R2 = st.number_input("Enter d1/R2:", value=0.0)
    r2_d2 = st.number_input("Enter r2/d2:", value=0.0)
    G = st.number_input("Enter G:", value=0.0)
    a1 = st.number_input("Enter a1:", value=0.0)
    a11 = st.number_input("Enter a11:", value=0.0)

    if st.button("Calculate Capacitance"):
        lambda_value, C_value = calculate_C(T, d1_R2, r2_d2, G, a1, a11)

        st.success(f" $\lambda$  (lambda): {lambda_value}")
        st.success(f"C (Capacitance): {C_value}  $\mu F$ ")

# Program 5: Astable multivibrator
def program_5():
    st.header("Program 5: Astable Multivibrator")

    def calculate_frequency(R, C):
        return 1 / (1.386 * R * C)

    n = st.number_input("Enter number of times you want to calculate:", value=1, min_value=1)

    for i in range(n):
        st.subheader(f"Calculation {i + 1}")
        R = st.number_input(f"Enter resistance (R) in ohms for calculation {i + 1}:", value=0.0)
        C = st.number_input(f"Enter capacitance (C) in farads for calculation {i + 1}:", value=0.0)

        if st.button(f"Calculate Frequency for Calculation {i + 1}"):
            frequency = calculate_frequency(R, C)
            st.success(f"Frequency: {frequency:.6f} Hz")

# Program 6: Air wedge
def program_6():
    st.header("Program 6: Air Wedge")

    def compute_thickness(lambda_val, l, beta, m):
        return (lambda_val * l * m) / (2 * beta)

    lambda_val = 5893e-10 # Wavelength in meters (5893 Å)

    l = st.number_input("Enter the distance between the wire (l) in meters:", value=0.0)
    beta = st.number_input("Enter the width of the ring ( $\beta$ ) in meters:", value=0.0)
    m = st.number_input("Enter the number of rings (m):", value=0)

    if st.button("Calculate Thickness"):
        thickness = compute_thickness(lambda_val, l, beta, m)
        st.success(f"Calculated thickness of wire (t): {thickness:.6e} meters")

# Program 7: Newton rings
def program_7():
    st.header("Program 7: Newton Rings")

    def compute_R(r_nm, r_n, m, wavelength):
        return (r_nm**2 - r_n**2) / (m * wavelength)

    wavelength = 5893e-10 # 5893 Å in meters

```



```

    r_n = st.number_input("Enter the radius of the nth ring in
meters:", value=0.0)
    r_nm = st.number_input("Enter the radius of the (n+m)th
ring in meters:", value=0.0)
    m = st.number_input("Enter the number of rings after nth
ring (m):", value=0)

    if st.button("Compute R"):
        R_value = compute_R(r_n, r_nm, m, wavelength)
        st.success(f"Computed R value: {R_value:.5e} m-1")

# Program 8: Minimum deviation
def program_8():
    st.header("Program 8: Minimum Deviation")

    def convert_to_decimal(degrees, minutes):
        return degrees + (minutes / 60)

    def calculate_coefficient_of_friction(D_deg, D_min,
A_deg, A_min):
        D_value = convert_to_decimal(D_deg, D_min)
        A_value = convert_to_decimal(A_deg, A_min)
        if A_value == 0:
            raise ValueError("A cannot be zero to avoid
division by zero.")
        D_radians = math.radians(D_value)
        A_radians = math.radians(A_value)
        mu = math.sin((D_radians + A_radians) / 2) /
math.sin(A_radians / 2)
        return mu

    n = st.number_input("Enter the number of calculations:",
value=1, min_value=1)

    results = []
    for i in range(n):
        st.subheader(f"Calculation {i + 1}")
        D_deg = st.number_input(f"Enter the degrees for D
(set {i + 1}):", value=0.0)
        D_min = st.number_input(f"Enter the minutes for D
(set {i + 1}):", value=0.0)
        A_deg = st.number_input(f"Enter the degrees for A
(set {i + 1}):", value=0.0)
        A_min = st.number_input(f"Enter the minutes for A
(set {i + 1}):", value=0.0)

        if st.button(f"Calculate Coefficient of Friction for
Calculation {i + 1}"):
            mu_result =
calculate_coefficient_of_friction(D_deg, D_min, A_deg,
A_min)
            results.append((D_deg, D_min, A_deg, A_min,
mu_result))
            st.success(f"mu = {mu_result:.6f}")

    if results:
        st.subheader("Final Results")

```

```

        for i, (D_d, D_m, A_d, A_m, mu) in
enumerate(results, 1):
            st.write(f"Calculation {i}:")
            st.write(f" D = {D_d}° {D_m}'")
            st.write(f" A = {A_d}° {A_m}'")
            st.write(f" -> mu = {mu:.6f}")
            st.write("-" * 30)

# Program 9: Small angle prism
def program_9():
    st.header("Program 9: Small Angle Prism")

    def deg_min_to_decimal(degrees, minutes):
        return degrees + (minutes / 60)

    def normal_incidence_method(A, d):
        A_rad = math.radians(A)
        d_rad = math.radians(d)
        return math.sin(A_rad + d_rad) / math.sin(A_rad)

    def normal_emergence_method(i, A):
        i_rad = math.radians(i)
        A_rad = math.radians(A)
        return math.sin(i_rad) / math.sin(A_rad)

    def minimum_deviation_method(D, A):
        D_rad = math.radians(D)
        A_rad = math.radians(A)
        return math.sin((D_rad + A_rad) / 2) / math.sin(A_rad
/ 2)

    method = st.selectbox("Choose a method to calculate the
refractive index:",
        ["Normal Incidence Method", "Normal
Emergence Method", "Minimum Deviation Method"])

    if method == "Normal Incidence Method":
        A_deg = st.number_input("Enter degrees for angle of
prism (A):", value=0.0)
        A_min = st.number_input("Enter minutes for angle of
prism (A):", value=0.0)
        A = deg_min_to_decimal(A_deg, A_min)

        d_deg = st.number_input("Enter degrees for angle of
deviation (d):", value=0.0)
        d_min = st.number_input("Enter minutes for angle of
deviation (d):", value=0.0)
        d = deg_min_to_decimal(d_deg, d_min)

        if st.button("Calculate Refractive Index"):
            mu = normal_incidence_method(A, d)
            st.success(f"Refractive Index (μ) using Normal
Incidence Method: {mu:.6f}")

    elif method == "Normal Emergence Method":
        i_deg = st.number_input("Enter degrees for angle of
incidence (i):", value=0.0)

```

```

i_min = st.number_input("Enter minutes for angle of
incidence (i):", value=0.0)
i = deg_min_to_decimal(i_deg, i_min)

A_deg = st.number_input("Enter degrees for angle of
prism (A):", value=0.0)
A_min = st.number_input("Enter minutes for angle of
prism (A):", value=0.0)
A = deg_min_to_decimal(A_deg, A_min)

if st.button("Calculate Refractive Index"):
    mu = normal_emergence_method(i, A)
    st.success(f"Refractive Index ( $\mu$ ) using Normal
Emergence Method: {mu:.6f}")

elif method == "Minimum Deviation Method":
    D_deg = st.number_input("Enter degrees for angle of
minimum deviation (D):", value=0.0)
    D_min = st.number_input("Enter minutes for angle of
minimum deviation (D):", value=0.0)
    D = deg_min_to_decimal(D_deg, D_min)

    A_deg = st.number_input("Enter degrees for angle of
prism (A):", value=0.0)
    A_min = st.number_input("Enter minutes for angle of
prism (A):", value=0.0)
    A = deg_min_to_decimal(A_deg, A_min)

    if st.button("Calculate Refractive Index"):
        mu = minimum_deviation_method(D, A)
        st.success(f"Refractive Index ( $\mu$ ) using Minimum
Deviation Method: {mu:.6f}")

# Program 10: Spectrometer i-i' curve
def program_10():
    st.header("Program 10: Spectrometer i-i' Curve")

    def dms_to_decimal(degrees, minutes):
        return degrees + (minutes / 60)

    def calculate_mu(D, A):
        i = (D + A) / 2
        A_rad = np.radians(A)
        i_rad = np.radians(i)
        if np.sin(A_rad / 2) == 0:
            return np.nan
        mu = np.sin(i_rad) / np.sin(A_rad / 2)
        return mu

    n = st.number_input("Enter the number of values:",
value=1, min_value=1)

    D_values = []
    A_values = []
    mu_values = []

    for i in range(n):
        st.subheader(f"Set {i + 1}")

```

```

D_deg = st.number_input(f"Enter D (Deviation
Angle) degrees for set {i + 1}:", value=0.0)
D_min = st.number_input(f"Enter D (Deviation
Angle) minutes for set {i + 1}:", value=0.0)
D = dms_to_decimal(D_deg, D_min)

A_deg = st.number_input(f"Enter A (Angle of Prism)
degrees for set {i + 1}:", value=0.0)
A_min = st.number_input(f"Enter A (Angle of Prism)
minutes for set {i + 1}:", value=0.0)
A = dms_to_decimal(A_deg, A_min)

mu = calculate_mu(D, A)
D_values.append(D)
A_values.append(A)
mu_values.append(mu)
st.success(f"Calculated Refractive Index ( $\mu$ ) for set {i
+ 1}: {mu}")

if st.button("Plot Results"):
    plt.figure(figsize=(5, 5))
    plt.scatter(D_values, mu_values, color='r', label='μ
values')
    plt.xlabel('D (Deviation Angle in Degrees)')
    plt.ylabel('μ (Refractive Index)')
    plt.title(f'Refractive Index Calculation for {n} Values')
    plt.legend()
    plt.grid()
    st.pyplot(plt)

# Program 11: Young's modulus (Koenig method)
def program_11():
    st.header("Program 11: Young's Modulus (Koenig
Method)")

    def youngs_modulus_koenig(frequency, length, width,
height, density):
        I = (width * height**3) / 12
        A = width * height
        E = (2 * math.pi * frequency)**2 * (density * A *
length**4) / I
        return E

    frequency = st.number_input("Enter the frequency of
vibration (Hz):", value=0.0)
    length = st.number_input("Enter the length of the beam
(m):", value=0.0)
    width = st.number_input("Enter the width of the beam
(m):", value=0.0)
    height = st.number_input("Enter the height of the beam
(m):", value=0.0)
    density = st.number_input("Enter the density of the
material (kg/m³):", value=0.0)

    if st.button("Calculate Young's Modulus"):
        E = youngs_modulus_koenig(frequency, length,
width, height, density)

```

```

        st.success(f"The Young's Modulus of the material is:
{E:.2e} Pa")

# Program 12: Young's modulus (Non-uniform bending)
def program_12():
    st.header("Program 12: Young's Modulus (Non-uniform
Bending)")

    def youngs_modulus(force, original_length,
change_in_length, area):
        stress = force / area
        strain = change_in_length / original_length
        E = stress / strain
        return E

    force = st.number_input("Enter applied force (N):",
value=0.0)
    original_length = st.number_input("Enter original length
(m):", value=0.0)
    change_in_length = st.number_input("Enter change in
length (m):", value=0.0)
    area = st.number_input("Enter cross-sectional area
(m²):", value=0.0)

    if st.button("Calculate Young's Modulus"):
        E = youngs_modulus(force, original_length,
change_in_length, area)
        st.success(f"Young's Modulus: {E} Pa")

# Program 13: Young's modulus (Stretching method)
def program_13():
    st.header("Program 13: Young's Modulus (Stretching
Method)")

    def youngs_modulus(F, L, A, delta_L):
        if delta_L == 0:
            return "Change in length cannot be zero"
        E = (F * L) / (A * delta_L)
        return E

    F = st.number_input("Enter the applied force (N):",
value=0.0)
    L = st.number_input("Enter the original length (m):",
value=0.0)
    A = st.number_input("Enter the cross-sectional area
(m²):", value=0.0)
    delta_L = st.number_input("Enter the change in length
(m):", value=0.0)

    if st.button("Calculate Young's Modulus"):
        E = youngs_modulus(F, L, A, delta_L)
        st.success(f"Young's Modulus: {E:.2e} Pa")

# Program 14: Young's modulus (Uniform bending)
def program_14():
    st.header("Program 14: Young's Modulus (Uniform
Bending)")

```

```

def youngs_modulus_uniform_bending(m, L, b, d, y):
    g = 9.81 # Acceleration due to gravity (m/s²)
    Y = (m * g * L**3) / (4 * b * d**3 * y)
    return Y

    m = st.number_input("Enter mass (kg):", value=0.0)
    L = st.number_input("Enter length of the beam (m):",
value=0.0)
    b = st.number_input("Enter breadth of the beam (m):",
value=0.0)
    d = st.number_input("Enter thickness of the beam (m):",
value=0.0)
    y = st.number_input("Enter depression (m):", value=0.0)

    if st.button("Calculate Young's Modulus"):
        Y = youngs_modulus_uniform_bending(m, L, b, d, y)
        st.success(f"Young's Modulus (Y) = {Y:.2e} N/m²")

# Program 15: Young's modulus (Cantilever)
def program_15():
    st.header("Program 15: Young's Modulus (Cantilever)")

    def youngs_modulus_cantilever(L, F, b, d, delta):
        E = (4 * L**3 * F) / (b * d**3 * delta)
        return E

    L = st.number_input("Enter the length of the cantilever
(m):", value=0.0)
    F = st.number_input("Enter the applied force (N):",
value=0.0)
    b = st.number_input("Enter the width of the beam (m):",
value=0.0)
    d = st.number_input("Enter the thickness (depth) of the
beam (m):", value=0.0)
    delta = st.number_input("Enter the deflection (m):",
value=0.0)

    if st.button("Calculate Young's Modulus"):
        E = youngs_modulus_cantilever(L, F, b, d, delta)
        st.success(f"Young's modulus (E) = {E:.2e}")

# Program 16: Absolute mutual inductance
def program_16():
    st.header("Program 16: Absolute Mutual Inductance")

    def calculate_lambda(a1, a11):
        return 2.303 * math.log(a1 / a11)

    def calculate_M(r, T, a1, a11, d1, d2):
        lambda_value = calculate_lambda(a1, a11)
        M = (r * T / (2 * math.pi)) * (1 + lambda_value / 2) *
(d1 / d2)
        return M

    r = st.number_input("Enter the fractional resistance (r):",
value=0.0)
    T = st.number_input("Enter the time in seconds (T):",
value=0.0)

```

```

    a1 = st.number_input("Enter the maximum distance from
center for one oscillation (a1):", value=0.0)
    a11 = st.number_input("Enter the maximum distance
from center for the eleventh oscillation (a11):", value=0.0)
    d1 = st.number_input("Enter the first kick in meters
(d1):", value=0.0)
    d2 = st.number_input("Enter the second kick in meters
(d2):", value=0.0)

    if st.button("Calculate Mutual Inductance"):
        M = calculate_M(r, T, a1, a11, d1, d2)
        st.success(f"The absolute mutual inductance (M) is:
{M}")

# Program 17: Comparison of mutual inductance
def program_17():
    st.header("Program 17: Comparison of Mutual
Inductance")

    def calculate_mutual_inductance_ratio(S1, S2, R1, R2):
        M1_M2_ratio = (S1 + R1) / (S2 + R2)
        return M1_M2_ratio

    S1 = st.number_input("Enter the resistance of the
secondary coil S1 (in ohms):", value=0.0)
    S2 = st.number_input("Enter the resistance of the
secondary coil S2 (in ohms):", value=0.0)
    R1 = st.number_input("Enter the resistance box value R1
(in ohms):", value=0.0)
    R2 = st.number_input("Enter the resistance box value R2
(in ohms):", value=0.0)

    if st.button("Calculate Ratio"):
        ratio = calculate_mutual_inductance_ratio(S1, S2, R1,
R2)
        st.success(f"The ratio of M1 to M2 is: {ratio}")

# Program 18: High resistance by leakage method
def program_18():
    st.header("Program 18: High Resistance by Leakage
Method")

    def calculate_resistance(C_microfarads, d1_meters,
d2_meters, t_seconds):
        C_farads = C_microfarads * 1e-6
        log_term = math.log10(d1_meters / d2_meters)
        R_ohms = t_seconds / (2.303 * C_farads * log_term)
        R_mega_ohms = R_ohms * 1e-6
        return R_mega_ohms

    C_microfarads = st.number_input("Enter capacitance (C)
in microfarads:", value=0.0)
    d1_meters = st.number_input("Enter distance (d1) in
meters:", value=0.0)
    d2_meters = st.number_input("Enter distance (d2) in
meters:", value=0.0)
    t_seconds = st.number_input("Enter time (t) in
seconds:", value=0.0)

```

```

    if st.button("Calculate Resistance"):
        R_mega_ohms = calculate_resistance(C_microfarads,
d1_meters, d2_meters, t_seconds)
        st.success(f"Resistance R: {R_mega_ohms} 10^6
(M)ohms")

# Program 19: Internal resistance of a cell BG
def program_19():
    st.header("Program 19: Internal Resistance of a Cell
BG")

    def calculate_internal_resistance(R, d1, d2):
        B = R * (d1 - d2) / d2
        return B

    R = st.number_input("Enter the resistance box value
(R):", value=0.0)
    d1 = st.number_input("Enter the kick in distance d1 (in
meters):", value=0.0)
    d2 = st.number_input("Enter the kick in distance d2 (in
meters):", value=0.0)

    if st.button("Calculate Internal Resistance"):
        B = calculate_internal_resistance(R, d1, d2)
        st.success(f"The internal resistance of the cell (B) is:
{B}")
44
# Program 20: Self inductance of a coil by Anderson
method
def program_20():
    st.header("Program 20: Self Inductance of a Coil by
Anderson Method")

    def calculate_self_induction(C, Q, R, S, M):
        L = C * (R * Q + M * (R + S))
        return L

    C = st.number_input("Enter the condenser value (C) in
microfarads:", value=0.0) * 1e-6
    Q = st.number_input("Enter the resistance box value Q
(in ohms):", value=0.0)
    R = st.number_input("Enter the resistance box value R
(in ohms):", value=0.0)
    S = st.number_input("Enter the resistance box value S
(in ohms):", value=0.0)
    M = st.number_input("Enter the resistance box value M
(in ohms):", value=0.0)

    if st.button("Calculate Self-Inductance"):
        L = calculate_self_induction(C, Q, R, S, M)
        st.success(f"The self-induction L is: {L} Henry")

# Main function to run the app
def main():
    st.title("Physics Programs")

```

```

program_choice = st.sidebar.selectbox(
    "Select a program:",
    [
        "Program 1: Polarimeter",
        "Program 2: Calibration of a High Range
Voltmeter",
        "Program 3: Sonometer",
        "Program 4: BG Absolute Capacity of a
Condenser",
        "Program 5: Astable Multivibrator",
        "Program 6: Air Wedge",
        "Program 7: Newton Rings",
        "Program 8: Minimum Deviation",
        "Program 9: Small Angle Prism",
        "Program 10: Spectrometer i-i' Curve",
        "Program 11: Young's Modulus (Koenig Method)",
        "Program 12: Young's Modulus (Non-uniform
Bending)",
        "Program 13: Young's Modulus (Stretching
Method)",
        "Program 14: Young's Modulus (Uniform
Bending)",
        "Program 15: Young's Modulus (Cantilever)",
        "Program 16: Absolute Mutual Inductance",
        "Program 17: Comparison of Mutual Inductance",
        "Program 18: High Resistance by Leakage
Method",
        "Program 19: Internal Resistance of a Cell BG",
        "Program 20: Self Inductance of a Coil by
Anderson Method"
    ]
)

if program_choice == "Program 1: Polarimeter":
    program_1()
elif program_choice == "Program 2: Calibration of a
High Range Voltmeter":
    program_2()
elif program_choice == "Program 3: Sonometer":
    program_3()
elif program_choice == "Program 4: BG Absolute
Capacity of a Condenser":
    program_4()
elif program_choice == "Program 5: Astable
Multivibrator":
    program_5()

```

```

elif program_choice == "Program 6: Air Wedge":
    program_6()
elif program_choice == "Program 7: Newton Rings":
    program_7()
elif program_choice == "Program 8: Minimum
Deviation":
    program_8()
elif program_choice == "Program 9: Small Angle
Prism":
    program_9()
elif program_choice == "Program 10: Spectrometer i-i'
Curve":
    program_10()
elif program_choice == "Program 11: Young's Modulus
(Koenig Method)":
    program_11()
elif program_choice == "Program 12: Young's Modulus
(Non-uniform Bending)":
    program_12()
elif program_choice == "Program 13: Young's Modulus
(Stretching Method)":
    program_13()
elif program_choice == "Program 14: Young's Modulus
(Uniform Bending)":
    program_14()
elif program_choice == "Program 15: Young's Modulus
(Cantilever)":
    program_15()
elif program_choice == "Program 16: Absolute Mutual
Inductance":
    program_16()
elif program_choice == "Program 17: Comparison of
Mutual Inductance":
    program_17()
elif program_choice == "Program 18: High Resistance
by Leakage Method":
    program_18()
elif program_choice == "Program 19: Internal Resistance
of a Cell BG":
    program_19()
elif program_choice == "Program 20: Self Inductance of
a Coil by Anderson Method":
    program_20()

if __name__ == "__main__":
    main()

```

This is the source code for the web-based python coding

CHAPTER 5

DIFFERENCES BETWEEN THE TWO CODES

The two Python scripts, `pythonwebapp.py` and `standalonepython.py`, serve similar purposes but are designed for different environments and user interactions.

5.1 User Interface:

- Web-based Python Programming is designed to be a web application using the **Streamlit** framework. It provides a Graphical User Interface (GUI) where users can interact with the program through a web browser. The interface includes input fields, buttons, and visual outputs like tables and graphs, making it user-friendly for those who prefer a web-based interaction.

- Standalone Python Programming is a standalone Python script intended to be run in a terminal or command-line interface (CLI). It relies on text-based input and output, where users enter data through the command line and receive results as text or simple plots. This script is more suitable for users comfortable with command-line interactions.

5.2 Input and Output Handling:

- In Web-based Python Programming, inputs are collected through Streamlit widgets like `st.number_input` and `st.button`, and outputs are displayed using Streamlit's `st.success`, `st.table`, and `st.pyplot` for visualizations. The web app dynamically updates based on user inputs, providing immediate feedback.

- In Standalone Python Programming, inputs are collected using the `input()` function, and outputs are printed to the console using `print()`. Graphs are displayed using `matplotlib.pyplot`, but the overall interaction is more static and requires re-running the script for new inputs.

5.3 Program Structure:

- Web-based Python Programming is structured around Streamlit's framework, with each program (e.g., Program 1, Program 2) encapsulated in a function and selected via a sidebar dropdown menu. The main function initializes the app and handles program selection.

- Standalone Python Programming is structured as a standalone script with a main function that prompts the user to select a program from a list. Each program is also encapsulated in a function, but the interaction flow is linear and text-based.

5.4 Error Handling and Validation:

- Web-based Python Programming relies on Streamlit's built-in widgets to handle user inputs, but it does not explicitly handle errors or invalid inputs in many cases. Users are expected to provide valid inputs.
- Standalone Python Programming includes more explicit error handling, especially in programs like Program 6 and Program 7, where it checks for valid inputs and provides feedback to the user if inputs are invalid.

5.5 Flexibility and Extensibility:

- Web-based Python Programming is more extensible for web-based applications, allowing for easy integration with other web technologies and deployment on cloud platforms. It is ideal for users who want to share their programs via a web interface.
- standalonepython.py is more flexible for local use and can be easily modified for batch processing or integration with other scripts. It is ideal for users who prefer running programs locally without a web interface

5.6 Key Differences:

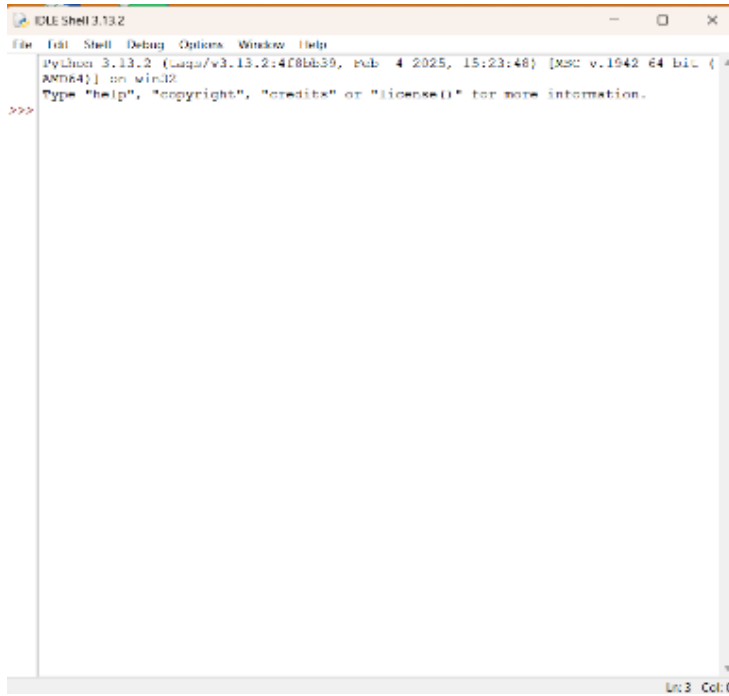
Feature	Standalone Python Programming	Web-Based Python Programming
Interface	Web-based (Streamlit)	Command-line interface (CLI)
User Interaction	GUI with input fields, buttons, and visual outputs	Text-based input/output via terminal
Input Handling	Streamlit widgets (<code>st.number_input</code> , <code>st.button</code>)	<code>input()</code> function
Output Handling	Streamlit outputs (<code>st.success</code> , <code>st.table</code> , <code>st.pyplot</code>)	<code>print()</code> function and <code>matplotlib</code> for graphs
Error Handling	Limited, relies on user input validation	Explicit error handling and input validation
Program Structure	Sidebar menu for program selection	Linear text-based program selection
Flexibility	Ideal for web-based applications and sharing	Ideal for local use and batch processing
Extensibility	Easily integrated with web technologies	Easily modified for local scripts and automation

CHAPTER 6

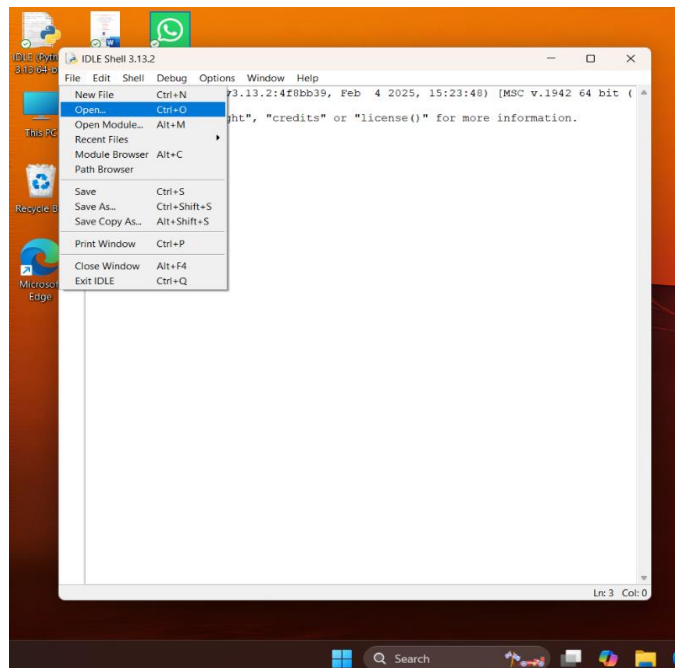
OUTPUTS OF BOTH CODES

6.1 STANDALONE PYTHON PROGRAM

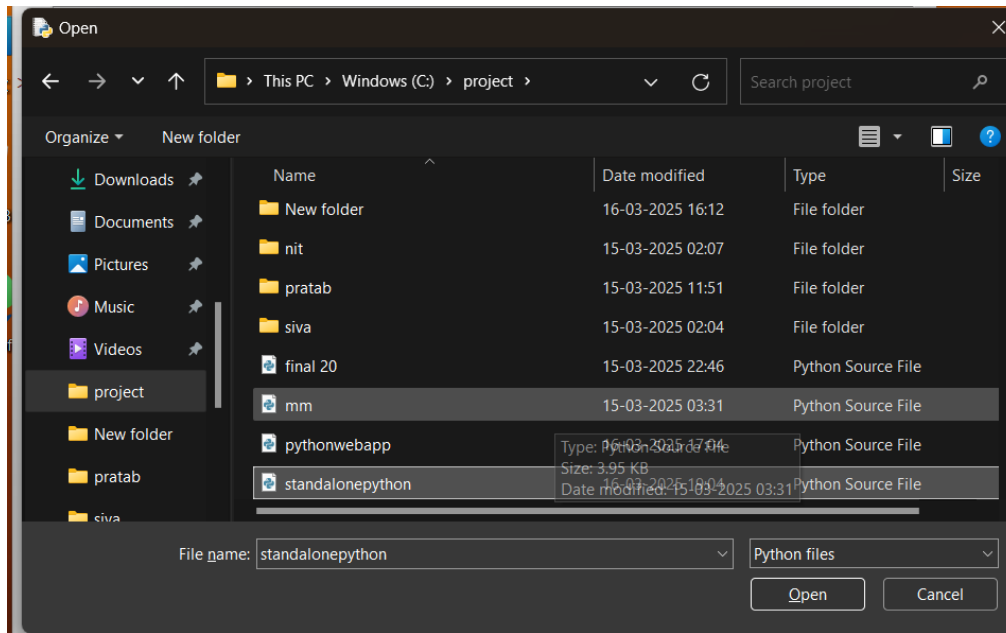
Open the saved code in python IDLE or in VS code, if you open IDLE this window



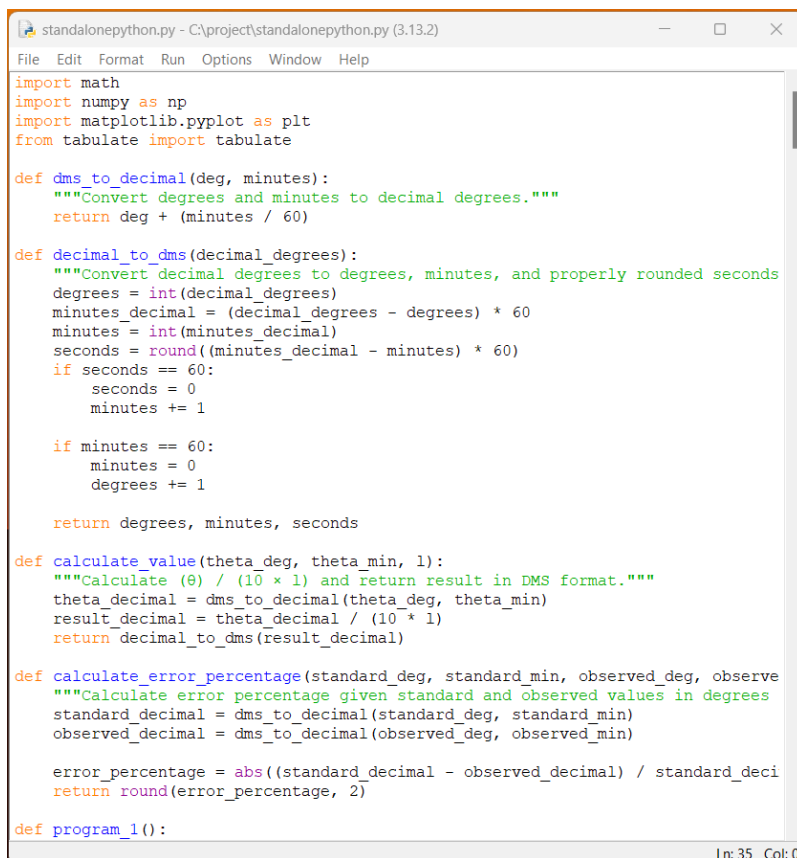
Here click file and open file, which opens the file manager, [you can use the shortcut (ctrl + o)]



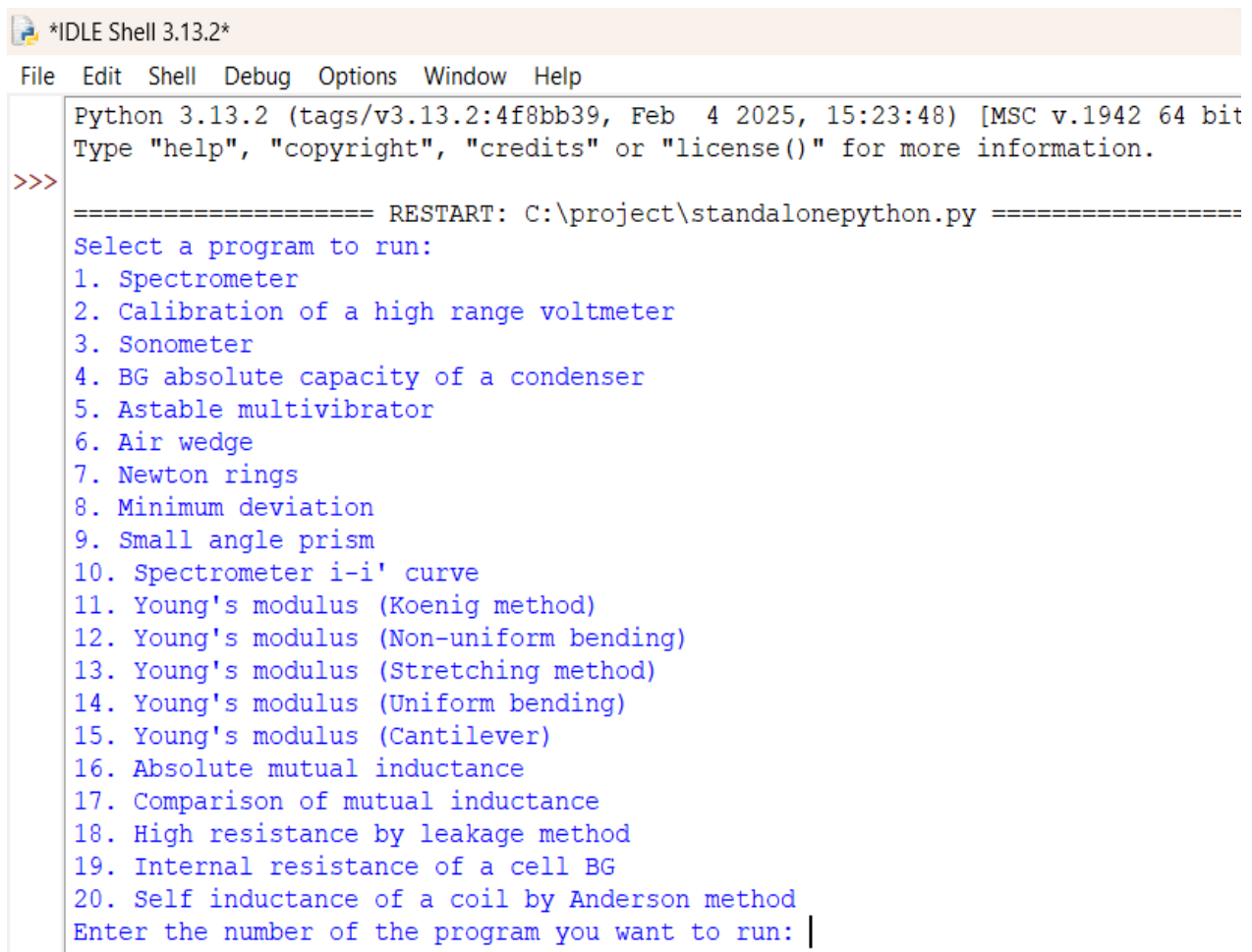
Then user selects then selects the desired program by the entering its value.



After clicking open, a tab opens which looks like the below picture,



Run the code by clicking F5 key in your keyboard. The output of the code will look like this,



```
*IDLE Shell 3.13.2*
File Edit Shell Debug Options Window Help
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\project\standalonepython.py =====
Select a program to run:
1. Spectrometer
2. Calibration of a high range voltmeter
3. Sonometer
4. BG absolute capacity of a condenser
5. Astable multivibrator
6. Air wedge
7. Newton rings
8. Minimum deviation
9. Small angle prism
10. Spectrometer i-i' curve
11. Young's modulus (Koenig method)
12. Young's modulus (Non-uniform bending)
13. Young's modulus (Stretching method)
14. Young's modulus (Uniform bending)
15. Young's modulus (Cantilever)
16. Absolute mutual inductance
17. Comparison of mutual inductance
18. High resistance by leakage method
19. Internal resistance of a cell BG
20. Self inductance of a coil by Anderson method
Enter the number of the program you want to run: |
```

Now enter the number of the program which

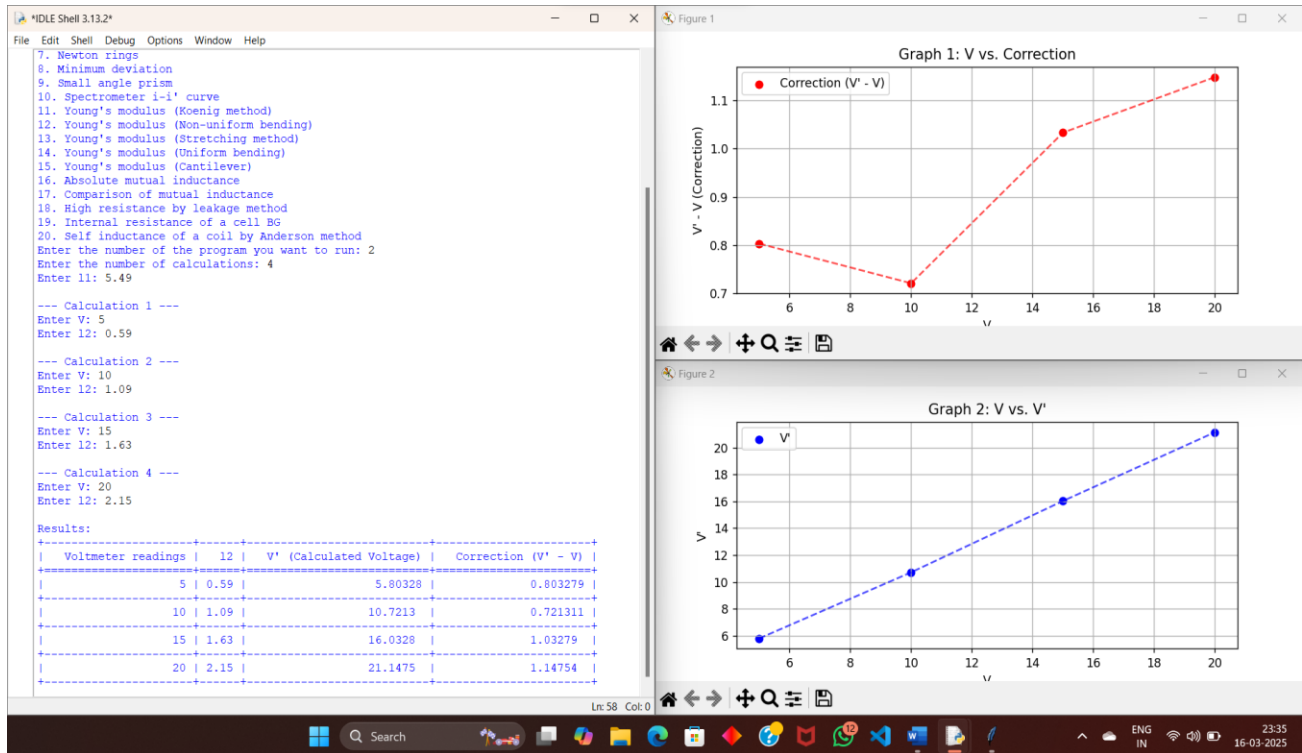
you want to perform. For example, if you want to perform calibration of a high range voltmeter, type 2 and click enter button.

It will ask the number of calculations to perform, for example enter 4, it will run the program to calculate BG Absolute capacity of a condenser.

Then it will ask the inputs,

enter the input values as you desired correctly, at last the program will run and shows you the output tabular form, it will also plot you two graphs (1. V vs correction, 2. V vs V' will show you the outputs which will also tabulate and plot your output.

It will show you like the images shown in the next page.



Like this you can calculate for the above given experiments.

6.2 WEB-BASED PYTHON PROGRAMMING

To get the output of this program you want to open your command prompt or VS code terminal, there you have to, use cd (change directory) to choose the location of the file it is been saved, then write this,

```

C:\Users\james>cd C:\project

C:\project>python -m streamlit run pythonwebapp.py

```

Here streamlit is the library, typing run, runs the file you type next, I have entered the name of the web-based program.

It will show results like this (shown below) and run the web-app in localhost, which is also shown below this

```
Command Prompt - python -m streamlit run pythonwebapp.py
Microsoft Windows [Version 10.0.22631.5039]
(c) Microsoft Corporation. All rights reserved.

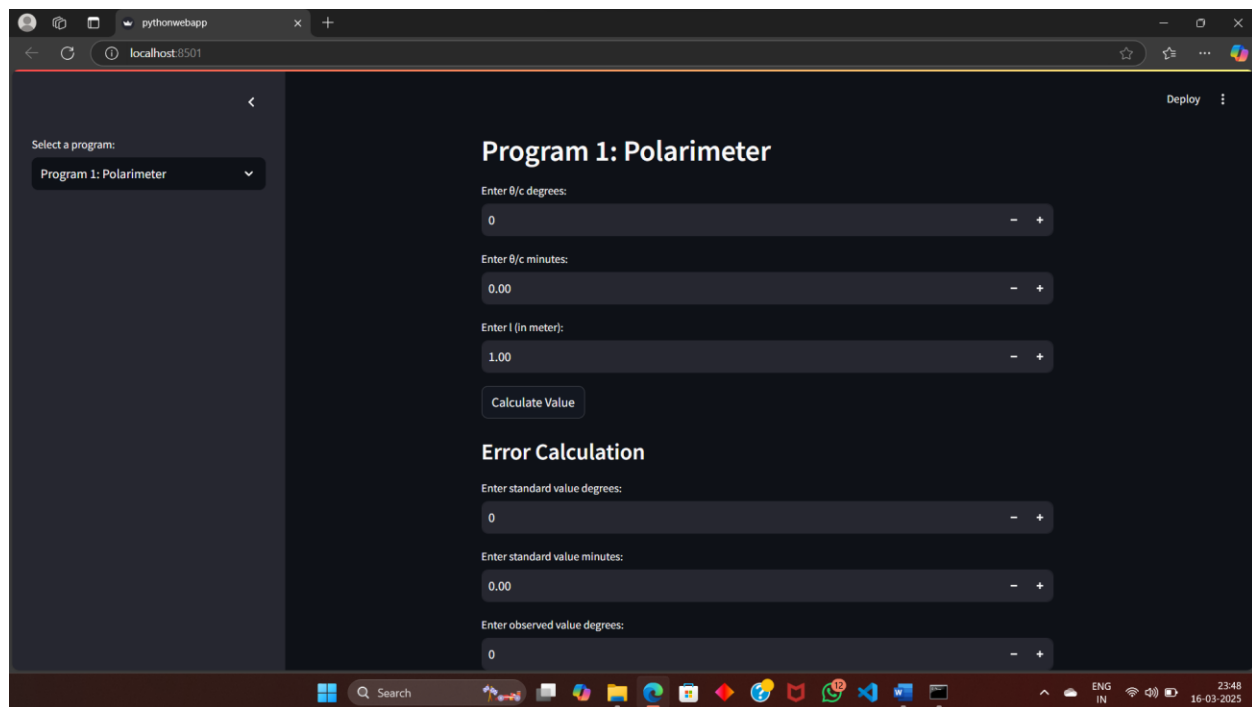
C:\Users\james>cd C:\project

C:\project>python -m streamlit run pythonwebapp.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.31.1:8501
```

This is how the interface of the web looks like, It needs internet connection to run and will only run on your IP address.



On the left, you can see Polarimeter, which shows a downward arrow clicking them, you can choose any of the twenty experiment you want to perform.

The twenty experiment available in this web is listed before.

<

Select a program:

Program 1: Polarimeter

Program 1: Polarimeter

Program 2: Calibration of a High Range Voltmeter

Program 3: Sonometer

Program 4: BG Absolute Capacity of...

Program 5: Astable Multivibrator

Program 6: Air Wedge

Program 7: Newton Rings

Program 8: Minimum Deviation

Physics Program

Program 1: Polarimeter

Enter θ/c degrees:

0

Enter θ/c minutes:

0.00

Enter l (in meter):

1.00

So if you want to perform polarimeter, choose the values ,

Program 1: Polarimeter

Enter θ/c degrees:

148

-

+

Enter θ/c minutes:

26.00

-

+

Enter l (in meter):

0.21

Press Enter to apply

-

+

Calculate Value

Error Calculation [↔](#)

Enter standard value degrees:

0

-

+

Enter standard value minutes:

0.00

-

+

Enter observed value degrees:

Click calculate value button, and enter values for the error calculation

Error Calculation [↗](#)

Enter standard value degrees:

 - +

Enter standard value minutes:

 - +

Enter observed value degrees:

 - +

Enter observed value minutes:

 - +

Calculate Error Percentage

Error Percentage: 6.63%

Final_pg_Report(1).pdf pythonwebapp

localhost:8501

Deploy

Select a program:

Program 2: Calibration of a ...

Enter V for calculation 2:

 - +

Enter I2 for calculation 2:

 - +

Calculation 3

Enter V for calculation 3:

 - +

Enter I2 for calculation 3:

 - +

Calculation 4

Enter V for calculation 4:

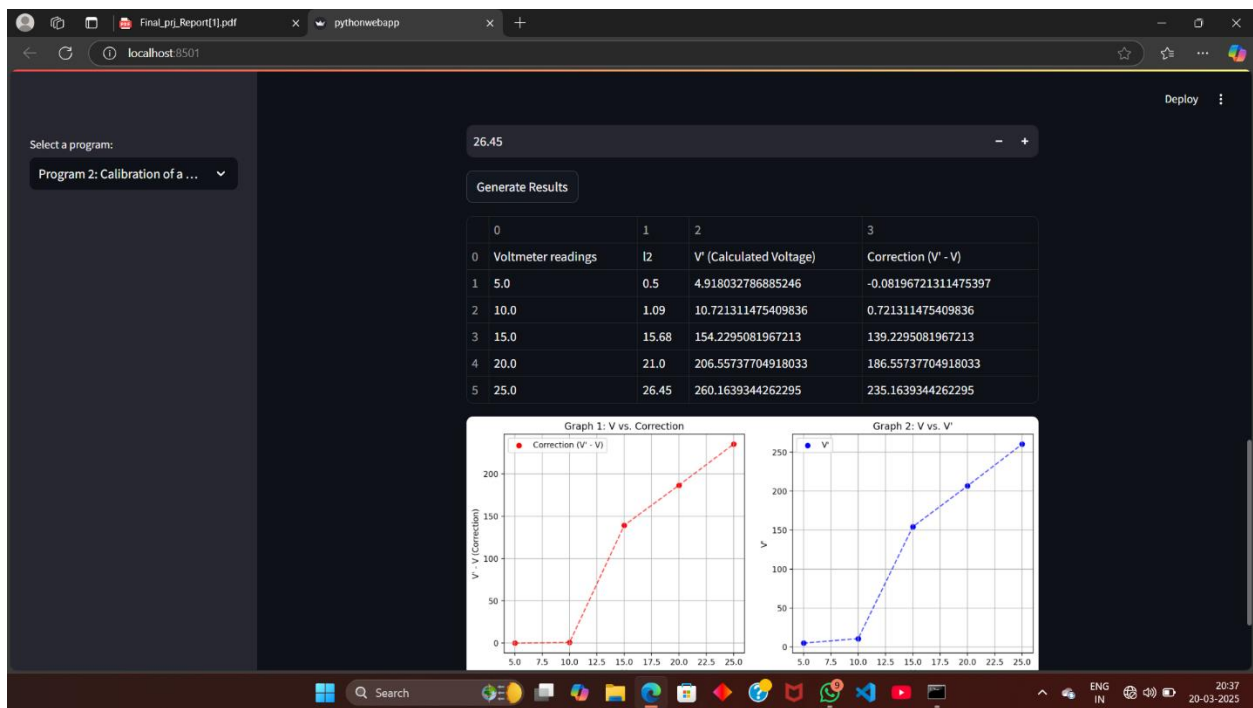
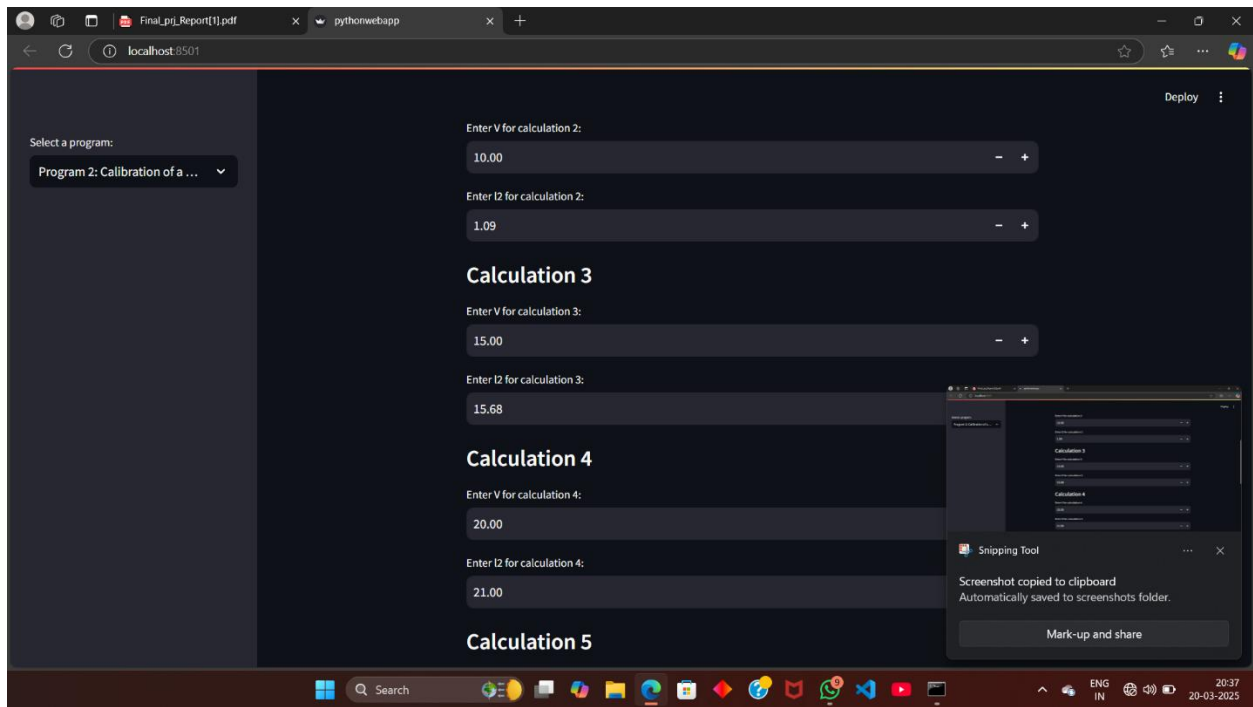
 - +

Enter I2 for calculation 4:

 - +

Calculation 5

20:37 20-03-2025



Thus, the numerical problems for the Physics experiments are solved.

CHAPTER 7

CONCLUSION

This project has been a great example of how Python programming can be used to solve numerical problems in physics experiments. By using Python's powerful libraries like NumPy, Tabulate, Matplotlib, and Streamlit, we created both a web application and standalone scripts that make it easy to perform calculations and analyze experimental data. The experiments we covered span a wide range of topics in physics, including optics, electromagnetism, mechanics, and material science, showing just how versatile Python can be for scientific computing.

What We Achieved:

Interactive Web App: A Streamlit-based web application was developed that's super user-friendly. You can input your experimental data, see the results right away, and even generate graphs without needing to know any programming.

Standalone Scripts: For those who prefer working in a terminal, we also created standalone Python scripts. These are perfect for running multiple calculations in one go and are great for repetitive tasks.

Wide Range of Experiments: We covered 20 different experiments, each with its own unique formula and implementation. This means Python can be used to solve a variety of physics problems, from calculating Young's modulus to analyzing interference patterns.

Visualizations: Matplotlib was used to create graphs that help you better understand your results. For example, you can plot V vs. Correction for voltmeter calibration or i - i' curves for spectrometer experiments.

Error Analysis: In experiments like the Polarimeter, we included error percentage calculations so you can check how accurate your measurements are and improves the experimental setup.

CHAPTER 8

REFERENCE

Python Programming

1. Van Rossum, G., & Drake, F. L. (2023). *The Python Language Reference*. Python Software Foundation. <https://docs.python.org/3/reference/>
(Official Python documentation)
2. Matthes, E. (2019). *Python Crash Course: A Hands-On, Project-Based Introduction to Programming* (2nd ed.). No Starch Press.
(Beginner-friendly Python guide)
3. McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.
(Covers NumPy and data handling)

Web Development with Python

4. Streamlit Team. (2023). *Streamlit Documentation*. <https://docs.streamlit.io/>
(Official guide for building web apps with Streamlit)
5. Richards, T. (2021). *Streamlit for Data Science: Create Interactive Data Apps in Python*. Packt Publishing.
(Practical Streamlit applications)
6. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2nd ed.). O'Reilly Media.
(For advanced web development concepts)

Numerical Computing & Visualization

7. Harris, C. R., et al. (2020). *Array Programming with NumPy*. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
(NumPy fundamentals)
8. Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment*. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
(Matplotlib introduction)
9. VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.
(Covers NumPy, Matplotlib, and Pandas)

10. Johansson, R. (2019). *Numerical Python: Scientific Computing and Data Science Applications with Numpy, SciPy, and Matplotlib* (2nd ed.). Apress.
(Advanced numerical methods)

Tabulate & Data Formatting

11. Tabulate GitHub Repository. (2023). *Python Tabulate Library*. <https://github.com/astanin/python-tabulate>
(Official documentation and examples)
12. PyPI. (2023). *Tabulate: Pretty-Print Tabular Data in Python*. <https://pypi.org/project/tabulate/>
(Installation and usage guide)

Physics & Computational Tools

13. Newman, M. E. J. (2013). *Computational Physics*. CreateSpace Independent Publishing.
(Numerical methods in physics)
14. Landau, R. H., Páez, M. J., & Bordeianu, C. C. (2015). *Computational Physics: Problem Solving with Python* (3rd ed.). Wiley.
(Python applications in physics)
15. Johansson, R. (2015). *Numerical Python: A Practical Techniques Approach for Industry*. Apress.
(Case studies in physics experiments)

Error Analysis & Validation

16. Taylor, J. R. (1997). *An Introduction to Error Analysis: The Study of Uncertainties in Physical Measurements* (2nd ed.). University Science Books.
(Essential for error handling in experiments)
17. Kiusalaas, J. (2013). *Numerical Methods in Engineering with Python* (3rd ed.). Cambridge University Press.
(Error validation techniques)

Additional Resources

18. Sweigart, A. (2019). *Automate the Boring Stuff with Python* (2nd ed.). No Starch Press.
(Practical Python scripting for automation)

19. Downey, A. B. (2015). *Think Python: How to Think Like a Computer Scientist* (2nd ed.). O'Reilly Media.
(Problem-solving with Python)
20. Slatkin, B. (2019). *Effective Python: 90 Specific Ways to Write Better Python* (2nd ed.). Addison-Wesley Professional.
(Best practices for clean code)
21. McKinney, W. (2017). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython* (2nd ed.). O'Reilly Media.
(Data analysis workflows)
22. W3Schools. (2023). Python Tutorial. <https://www.w3schools.com/python/>
(Beginner-friendly syntax examples)
23. Mozilla Developer Network (MDN). (2023). Web Development Basics. <https://developer.mozilla.org/>
(For web app fundamentals)
24. Pérez, F., & Granger, B. E. (2007). IPython: A System for Interactive Scientific Computing. *Computing in Science & Engineering*, 9(3), 21–29. <https://doi.org/10.1109/MCSE.2007.53>
(Interactive computing in Python)
25. Virtanen, P., et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17(3), 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
(SciPy for advanced computations)